



数学建模暑期集训

建模算法问题概述

成都理工大学 卢鹏

2026年7月16日

精勤求學 敦篤勵志 果毅力行 忠恕任事



内容提要

CONTENT

1

算法设计与**效率**（意识）

2

现代优化算法（**启发式**方法）

3

机器学习算法概述

4

算法改进举例

5

复杂问题求解**策略**

5

算法写作与**要点**



1. 碎球问题-**Google**面试题

算法设计

问题：有一栋100层高的大楼，给你两个完全相同的玻璃球。假设从某一层开始，丢下玻璃球会摔碎。那么怎么利用手中的**两个球**，用什么**最优策略**知道这个临界的层是第几层？

问题：什么是最优策略？

目标1：发生最坏情况时的结果最好。

目标2：当实验次数足够多，平均每次找到临界层的次数最少。





1. 碎球问题-Google面试题

算法设计与模拟

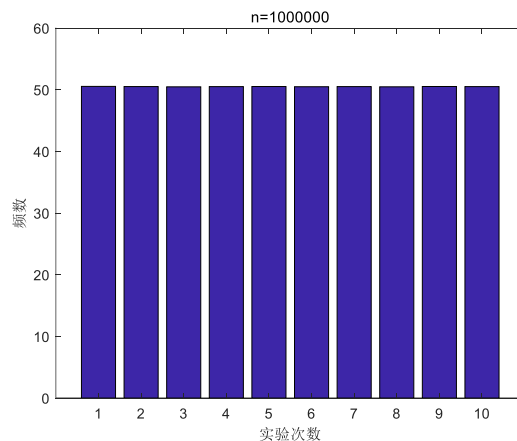
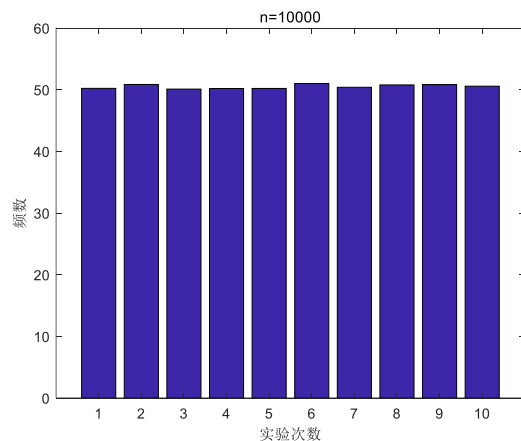
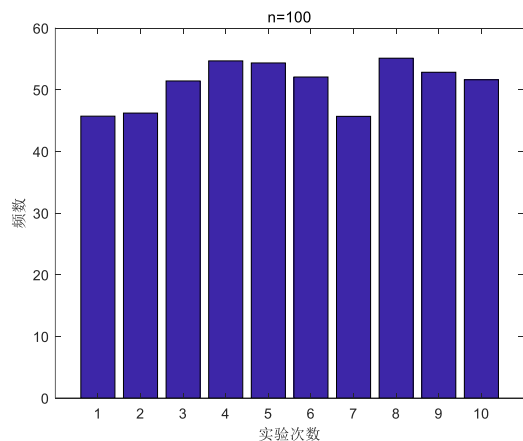
方法1：穷举法，一层一层尝试，按照1~100顺序尝试，

目标1-最坏情况测100次。

目标2-1 理论的数学期望 = 50.5次。

$$P(A) = \frac{1}{100} (1 + 2 + 3 + \cdots + 100) = 50.5$$

目标2-2 产生1~100之间的一个随机整数，模拟整个过程。



结果说明：

实验次数1百次，
频数为51.3690次。
实验次数1万次，
频数为50.4062次。
实验次数1百万次，
频数为50.4996次。



1. 碎球问题-Google面试题

算法设计与模拟

方法2：二分法，比如从50层开始尝试，若没碎则找75层，按此下去50 → 75 → $[75+25/2]=87$ → $[87+13/2]=93$ → $[93+7/2]=96$ → $[96+4/2]=98$ → $[98+2/2]=99$ → 100 最坏情况测26次；

但是如果50层碎了，剩一个球，需要尝试1~49层，这时候的最坏情况测49+1=50次。

目标1-最坏情况测50次。

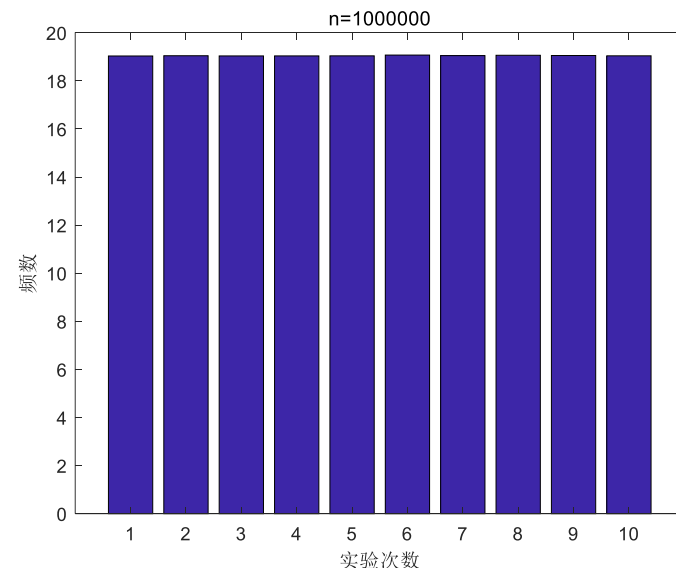
目标2-1 理论的数学期望 = 19.04次。

$$P(A) = \frac{1}{100}(2+3+\cdots+50+50) + \frac{1}{100}(3+4+\cdots+26+26) + \frac{1}{100}(4+5+\cdots+14+14) \\ + \frac{1}{100}(5+6+\cdots+9+9) + \frac{1}{100}(6+7+7) + \frac{1}{100}(7+7) + \frac{1}{100}(7) + \frac{1}{100}(8) = 19.04$$

目标2-2 产生1~100之间的一个随机整数，模拟整个过程1百万次。

结果说明：频数约为19.0465次。

评注：方法有一定提升，但不是最优。





1. 碎球问题-Google面试题

算法设计与模拟

方法3：粗调、细调法

粗调：调10位数 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100，最坏情况测10次。

细调：调个位数，比如30没碎，40碎了，则 31 → 32 → ... → 39，最坏情况测9次。

目标1-最坏情况测10+9=19次。

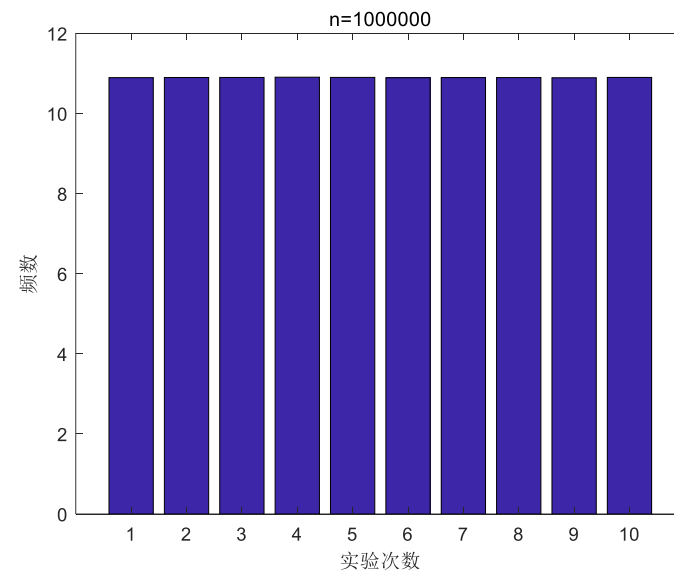
目标2-1 理论的数学期望 = 10.9次。

$$P(A) = \frac{1}{100} [(2 + 3 + \dots + 10 + 10) \times 10 + 10 + 20 + \dots + 90] = 10.9$$

目标2-2 产生1~100之间的一个随机整数，模拟整个过程1百万次。

结果说明：频数约为10.8974次。

评注：虽然最坏情况降低为10+9=19次，但目标1也不是最优。





1. 碎球问题-Google面试题

方法4: 最优算法 (粗细条尽量均匀化)

第一次扔 k 层, 则次数 $\text{time}=1$, 第二次如果破了, 要试从1到 $k-1$ 层,此时需要 $\text{sumTime}=\text{time}+k-1=k$ 次; 如果没破, 还要从 $2k-1$ 层扔, 则次数为 $\text{time}=2$, 如果破了, 还需要扔 $k+1$ 到 $2k-2$ 层, 再加上前面扔的两次 $\text{time}=2$, 即 $\text{sumTime}=\text{time}+k-2=k$, 还是 k 次。注意每多扔一次少测试一层。

如果 $2k-1$ 层没破, 还需要扔 $2k$ 到 $3k-3$ 层, 再加上前面扔的两次 $\text{time}=3$, 即 $\text{sumTime}=\text{time}+k-3=k$ 。还是 k 次.那么假设扔 k 次能知道临界点, 共有 n 层楼。以此类推如果满足

$k + (k-1) + (k-2) + (k-3) + \dots + (k-(k-1)) \geq n$ 。可以化简得到: $k(k+1) \geq 2n$

这里 $n=100$, 所以解得 $k=14$ 。所以只要14次就可以确认那层试临界层。

目标1: 14 → 27 → 39 → 50 → 60 → 69 → 77 → 84 → 90 → 95 → 99 → 100, 最坏情况测14次。

评注: 本方式利用方法3的思想, 粗调非均匀, 且各种结果的最坏情况都一样 (除了右边界)。



1. 碎球问题-Google面试题

目标2-1：理论的数学期望 = 10.35次。

$$P(A) = \frac{1}{100} [(2+3+\cdots+14+14) + (3+4+\cdots+14+14) + \cdots + (12+13+14+14) + 12] = 10.35$$

目标2-2 产生1~100之间的一个随机整数，模拟整个过程1百万次。

结果说明：频数约为10.3490次。

评注：两种目标下的搜索次数比较

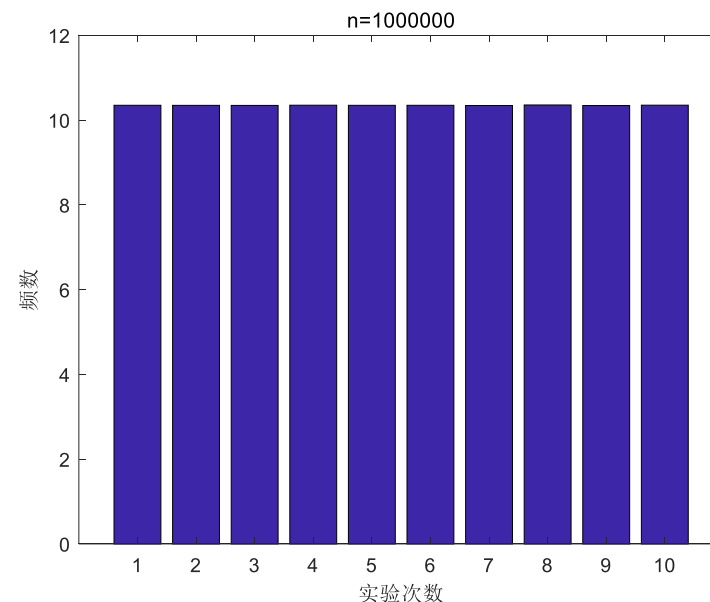
目标1：方法1 (100次) > 方法2 (50次) > 方法3 (19次) > 方法4 (14次)

目标2：方法1 (50.5次) > 方法2 (19.04次) >

方法3 (10.9次) > 方法4 (10.35次)

思考：方法4在目标1下是最优的，请问在目标2下是否是最优？

你们能从中获得什么体会？





1. 碎球问题-进一步延伸（简单→复杂）

思考：如果手中是三个球，用什么最优策略知道这个临界
的层是第几层？只考虑发生最坏情况时的结果最好。



策略：第一颗球使用方法2（二分法），如果碎了，则剩下的两颗球使用方法4，**最坏情况测1+10=11次。**（**意识：**复杂问题可使用混合方法）

说明：如果是四个球，则前2颗球使用方法2，如果都碎了，则剩下的两颗球使用方法4，**最坏情况测1+1+7=9次。**若前两颗球使用三分法，**最坏情况测1+1+8=10次。**

原因：这是因为二分法两次将区间缩短为1/4，三分法只能将区间缩短为1/3.

问题-1：如果球无限多，则次数为n次，为 $2^{n-1} \geq 100$ ，二分法测8次。还能提升吗？

问题-2：如果最多只能测8次，请问最少需要几颗球？



2. 现代优化算法

最优算法有时是有效的，也有些却十分麻烦，计算量非常大，以至于实际上是不可行的。实际求解问题要考虑经济效果，最优难以达到，次最优已经足够好的了。这就考虑采用**现代优化算法（启发式算法）**。分析人员必须运用自己的感知和洞察力，从与其有关而较基本的模型及算法中寻求其间的联系，从中得到启发，去发现适于解决该问题的思路和途径，这种方法称为启发式方法，由此建立的算法称为启发式算法。

比较著名的启发式算法有遗传算法、模拟退火算法、神经网络算法、粒子群算法、蚁群算法、禁忌搜索算法、拉格朗日松弛算法、混合优化算法等。



2.1 遗传算法 (GA算法)

遗传算法 (Genetic Algorithms, 简称 GA) 是一种基于自然选择原理和自然遗传机制的搜索 (寻优) 算法, 它是模拟自然界中的生命进化机制, 在人工系统中实现特定目标的优化。遗传算法的实质是通过群体搜索技术, 根据**适者生存**的原则**逐代进化**, 最终得到最优解或准最优解。它必须做以下操作: 初始群体的产生、求每一个体的适应度、根据适者生存的原则选择优良个体、被选出的优良个体两两**配对**, 通过随机**交叉**其染色体的基因并随机**变异**某些染色体的基因后生成下一代群体, 按此方法使群体逐代进化, 直到满足进化终止条件。



2.1 遗传算法的实现步骤

1. 编码

遗传算法的编码有浮点编码和二进制编码两种。

二进制编码：设某一参数的取值范围为 (L, U) ，使用长度为 k 的二进制编码表示该参数，则它共有 2^k 种不同的编码，该参数编码时的对应关系为

$$0000000000000000=0 \rightarrow L$$

$$0000000000000001=1 \rightarrow L + \delta$$

$$0000000000000010=2 \rightarrow L + 2 \delta$$

$$0000000000000011=3 \rightarrow L + 3 \delta$$

.....

$$1111111111111111=2^k-1 \rightarrow U$$

易知：

$$\delta = \frac{U - L}{2^k - 1}$$



2.1 遗传算法的实现步骤

2. 解码

二进制数据还原成十进制，设某一个体的二进制编码为 $b_k b_{k-1} b_{k-2} \cdots b_3 b_2 b_1$ ，则对应的解码公式为

$$x = L + \left(\sum_{i=1}^k b_i 2^{i-1} \right) \frac{U - L}{2^k - 1}$$

例如，设有参数 $x \in [2, 4]$ ，现用5位二进制数对 x 进行编码，可得 $2^5=32$ 条染色体：

00000, 00001, 00010, 00011, 00100, 00101, 00110, 00111

01000, 01001, 01010, 01011, 01100, 01101, 01110, 01111

10000, 10001, 10010, 10011, 10100, 10101, 10110, 10111

11000, 11001, 11010, 11011, 11100, 11101, 11110, 11111



2.1 遗传算法的实现步骤

例如：10101，它对应的十进制值为：

$$\sum_{i=1}^k b_i 2^{i-1} = 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 0 \times 2^3 + 1 \times 2^4 = 21$$

运用解码公式，则对应参数的x值为

$$x = 2 + 21 \times \frac{4-2}{2^5-1} = 3.3548$$

遗传算法的编码和解码：

在宏观上可以对应生物的基因型和表现型；

在微观上可以对应DNA的转录和翻译两个过程。

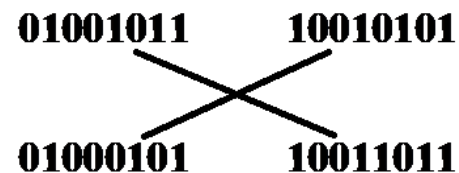


2.1 遗传算法的实现步骤

3. 交配

“交配运算”是使用单点或多点进行交叉的算子。首先用随机数产生一个或多个交配点位置，然后两个个体在交配点位置互换部分基因码，形成两个子个体。

例如：有两条染色体 $S_1=01001011$, $S_2=10010101$ 交换其后4位基因，如图所示： $S_1^*=01000101$, $S_2^*=10011011$ 可以被看做是原染色体 S_1 , S_2 的子代染色体。



4. 突变

“突变运算”是使用基本位进行基因突变。为了避免在算法迭代后期出现种群过早收敛，对于二进制的基因码组成的个体种群，实行基因码的小几率翻转，对于二进制编码即0变为1，而1变为0。

例如：将染色体 $S=11001101$ 第三位上的0变为1，即
 $S=11001101 \rightarrow S^*=11101101$
 S^* 可以被看做是原染色体 S 的子代染色体。



2.1 遗传算法的实现步骤

5. 倒位

除了交配和突变之外，复杂问题可能需要“倒位”。染色体上的区间可能一次又一次发生倒位，且通过自交出现不同的倒位纯合体，致使它们与其原来的物种不能交配，形成生殖隔离，结果产生新族群或变种。

例如： $S=1001011011101110011010101001$
 $\rightarrow S^*=1001011001011001110111101001$

6. 个体适应度评估

遗传算法依照与个体适应度成正比的几率决定当前种群中各个个体遗传到下一代群体中的机会。个体适应度大的个体更容易被遗传到下一代。通常，求目标函数的最大值的问题可以直接把目标函数作为检测个体适应度大小的函数。



2.1 遗传算法的实现步骤

7. 复制

复制运算是根据个体适应度大小决定其下代遗传的可能性。若设种群中个体总体为 N ，个体 i 的适应度为 f_i ，被选取的几率：

$$P_i = \frac{f_i}{\sum_{k=1}^N f_k}$$

当个体复制的几率决定后，再产生 $[0, 1]$ 区间的均匀随机数来决定哪些个体参加复制。若个体适应度高，则被选取的几率 P_i 就大，且可能被多次选中，它的遗传基因就会在种群中扩散；若个体的复制几率小，则会被逐渐淘汰。

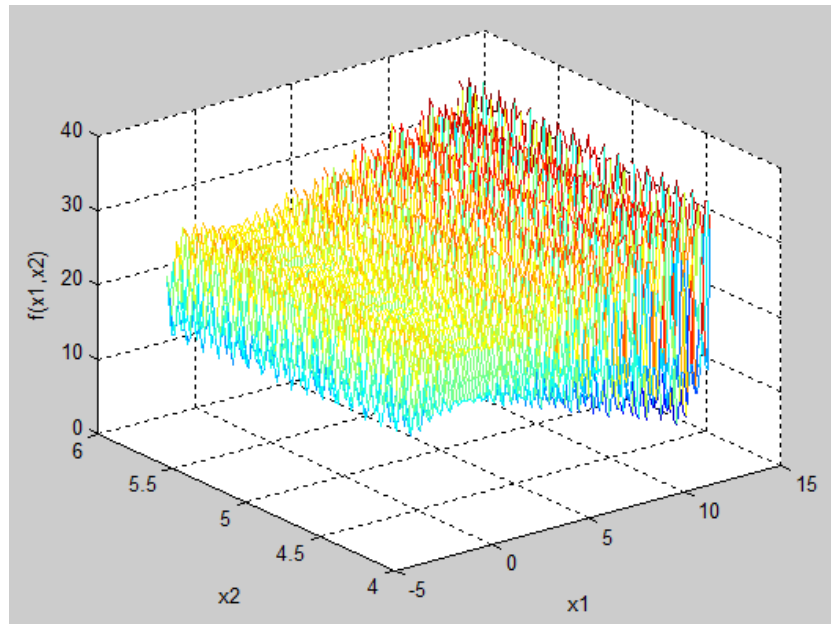


2.1 遗传算法作业过程

例1. 求 $\max f(x_1, x_2) = 21.5 + x_1 \sin(4\pi x_1) + x_2 \sin(20\pi x_2)$

$$s.t. \begin{cases} -3.0 \leq x_1 \leq 12.1 \\ 4.1 \leq x_2 \leq 5.8 \end{cases}$$

函数的三维图形如图所示：





2.1 遗传算法作业过程

1. 编码与解码

若变量的区间为 (L, U) ，假定要求的精度是小数点后4位，则每个变量被分成至少 $(U - L) \times 10^4$ 个部分。对一个变量的二进制串位数用以下公式计算：

$$2^{m_j-1} < (U - L) \times 10^4 \leq 2^{m_j} - 1$$

本题中，两个自变量所构成的染色体数串可以表示如下：

$$\begin{cases} -3 \leq x_1 \leq 12.1, & 2^{m_1-1} < [12.1 - (-3)] \times 10^4 = 151000 \leq 2^{m_1} - 1 \\ 4.1 \leq x_2 \leq 5.8, & 2^{m_2-1} < [5.8 - 4.1] \times 10^4 = 17000 \leq 2^{m_2} - 1 \\ m_1 = 18, m_2 = 15, m = m_1 + m_2 = 18 + 15 = 33 \end{cases}$$



2.1 遗传算法作业过程

本例中任一染色体数串都是33位，即000001010100101001101111011111110。
以上编码前18位表示 x_1 ，后15位表示 x_2 ，如表格所示：

自变量	实际值	十进制	实际值
x_1	000001010100101001	5417	-2.687969
x_2	101111011111110	24318	5.361653

$$x_1 = -3.0 + 5417 \times \frac{12.1 - (-3)}{2^{18} - 1} = -2.687969$$

$$x_2 = 4.1 + 24318 \times \frac{5.8 - 4.1}{2^{15} - 1} = 5.361653$$



2.1 遗传算法作业过程

假设初始群中有10个个体，其染色体可随机生成如下：

U1=000001010100101001	1011110111111110
U2=001110101110011000	000010101001000
U3=111000111000001000	010101001000110
U4=100110110100101101	000000010111001
U5=000010111101100010	001110001101000
U6=111110101011011000	000010110011001
U7=110100010111110001	001100111011101
U8=001011010100001100	010110011001100
U9=111110001011101100	011101000111101
U10=111101001110101010	000010101101010

相对于十进制的实际值为：

U1=[-2. 687969	5. 361653]	U2=[0. 474101	4. 170144]
U3=[10. 419457	4. 661461]	U4=[6. 159951	4. 109598]
U5=[-2. 301286	4. 477282]	U6=[11. 788084	4. 174346]
U7=[9. 342067	5. 121702]	U8=[-0. 330256	4. 694977]
U9=[11. 671267	4. 873501]	U10=[11. 446273	4. 171908]



2.1 遗传算法作业过程

2. 评价个体适应度

对于一个染色体数串的适应度的评价由下列三个步骤组成：

(1) 将染色体串进行解码，转化成真实值，即

$$\mathbf{x}^k = (x_1^k, x_2^k), \quad k = 1, 2, 3, \dots$$

(2) 评价目标函数 $f(\mathbf{x}^k)$

(3) 将目标函数值转化为适应度。对于极大值问题，适应度可作为目标函数值：

$$\text{eval}(U_k) = f(\mathbf{x}_k), \quad k = 1, 2, 3, \dots$$



2.1 遗传算法作业过程

上述染色体的适应度值如下：

$$\mathit{eval}(U_1) = f(-2.687969, 5.361653) = 19.805119$$

$$\mathit{eval}(U_2) = f(0.474101, 4.170144) = 17.370896$$

$$\mathit{eval}(U_3) = f(10.419457, 4.661461) = 9.590546$$

$$\mathit{eval}(U_4) = f(6.159951, 4.109598) = 29.406122$$

$$\mathit{eval}(U_5) = f(-2.301286, 4.477282) = 15.686091$$

$$\mathit{eval}(U_6) = f(11.788084, 4.174346) = 11.900541$$

$$\mathit{eval}(U_7) = f(9.342067, 5, 121702) = 17.958717$$

$$\mathit{eval}(U_8) = f(-0.330256, 4.694977) = 19.763190$$

$$\mathit{eval}(U_9) = f(11.671267, 4.873501) = 26.401669$$

$$\mathit{eval}(U_{10}) = f(11.446273, 4.171908) = 10.252480$$



2.1 遗传算法作业过程

依照染色体的适应度值进行新种群的复制，步骤如下：

(1) 计算染色体 U_k 的适应度值

$$eval(U_k) = f(x^k), \quad k = 1, 2, 3, \dots$$

(2) 计算种群的适应度值总和

$$F = \sum_{k=1}^{pop_size} eval(U_k)$$

(3) 计算每个染色体被复制的概率

$$P_k = \frac{eval(U_k)}{F}$$

(4) 计算每个染色体被复制的累积概率

$$Q_k = \sum_{j=1}^k P_k$$



2.1 遗传算法作业过程

3. 新种群复制

根据以上的计算方法，可以先计算出种群中每个染色体的适应度和（累计）概率，如下表：

染色体	适应度	被复制的概率	被复制的累计概率
U1	19.805119	0.111180	0.111180
U2	17.370896	0.097515	0.208695
U3	9.590546	0.053839	0.262534
U4	29.406122	0.165077	0.427611
U5	15.686091	0.088057	0.515668
U6	11.900541	0.066806	0.582475
U7	17.958717	0.100815	0.683290
U8	19.763190	0.110945	0.794234
U9	26.401669	0.148211	0.942446
U10	10.252480	0.057544	1.000000

□



2.1 遗传算法作业过程

依照轮盘选择法，随机产生0~1的随机数10个，如下：

0.301431 0.322062 0.766503 0.881893 0.350871
0.538392 0.177618 0.343242 0.032685 0.197577

第一个随机数为0.301431，大于Q3小于Q4，所以U4被选中

第二个随机数为0.322062，大于Q3小于Q4，所以U4再次被选中

第三个随机数为0.766503，大于Q7小于Q8，所以U8被选中

.....

第十个随机数为0.197577，大于Q1小于Q2，所以U2被选中



2.1 遗传算法作业过程

依照轮盘选择法，新种群的染色体组成如下：

U1=100110110100101101	000000010111001 (U4)
U2=100110110100101101	000000010111001 (U4)
U3=001011010100001100	010110011001100 (U8)
U4=111110001011101100	011101000111101 (U9)
U5=100110110100101101	000000010111001 (U4)
U6=110100010111110001	001100111011101 (U7)
U7=001110101110011000	000010101001000 (U2)
U8=100110110100101101	000000010111001 (U4)
U9=000001010100101101	101110111111110 (U1)
U10=001110101110011000	000010101001000 (U2)

选择机理：染色体的适应度大意味着 $[Q_k, Q_{k+1}]$ 区间跨度大，随机数发生器产生的均匀随机数就会有更大的概率落在较大长度的 $[Q_k, Q_{k+1}]$ 区间里，这样具有较大 P_k 值的染色体自然更有机会复制到下一代。



2.1 遗传算法作业过程

4. 新种群交配

(1) 交配染色体数量的确定

交配染色体的数量等于染色体总量乘以交配概率。假定本题中交配概率 $P_c=0.2$ ，则参加交配的染色体数量为2条。

(2) 交配染色体对象的确定

随机产生两个0~1之间的随机数，如：0.456732，0.623781其对应着累计概率的区间为 $[Q_4, Q_5]$ ， $[Q_6, Q_7]$ ，则选择选择U5，U7进行交配。



2.1 遗传算法作业过程

(3) 在交配池发生交配

交配的种类有单点交配和多点交配，这里取单点交配。

交配点的选择以随机数产生，计算机随机生成一个介于1~32的整数（染色体长度为33）。假设所产生的整数为1，那么自1位置以后开始分割交配，得到新的子辈染色体。

U5=1 00110110100101101 000000010111001

U7=0 01110101110011000 000010101001000

交配后→

U5=1 01110101110011000 000010101001000

U7=0 00110110100101101 000000010111001



2.1 遗传算法作业过程

5. 基因突变

染色体串基因突变位置的确定

假定本题中突变概率 $P_c=0.01$ ，共 $33 \times 10=330$ 基因，则希望每一代中有3.3个突变基因，每个基因的突变几率是均等的。因此，可以产生330个介于0~1之间的随机数（需编号），然后将小于0.01者选出，并将其对应的基因值加以翻转，如下表所示：

基因位置↵	染色体位置↵	基因位置↵	随机数↵	↵
105/33=3...6↵	3+1=4↵	6↵	0.009857↵	↵
164/33=4...32↵	4+1=5↵	32↵	0.003113↵	↵
199/33=6...1↵	6+1=7↵	1↵	0.000946↵	↵
329/33=9...32↵	9+1=10↵	32↵	0.001282↵	↵



2.1 遗传算法作业过程

在突变后，最终新种群的染色体组如下：

U1=100110110100101101	000000010111001
U2=100110110100101101	000000010111001
U3=001011010100001100	010110011001100
U4=111111001011101100	011101000111101
U5=101110101110011000	000010101001010
U6=110100010111110001	001100111011101
U7=100110110100101101	000000010111001
U8=100110110100101101	000000010111001
U9=000001010100101101	101110111111110
U10=001110101110011000	000010101001010



2.1 遗传算法作业过程

新一代的相对应实际值和适应度值如下：

$$eval(U_1) = f(6.159951, 4.109598) = 29.406122$$

$$eval(U_2) = f(6.159951, 4.109598) = 29.406122$$

$$eval(U_3) = f(-0.330256, 4.694977) = 19.763190$$

$$eval(U_4) = f(11.907206, 4.873501) = 5.702781$$

$$eval(U_5) = f(8.024130, 4.170248) = 19.910251$$

$$eval(U_6) = f(9.342067, 5.117020) = 17.958717$$

$$eval(U_7) = f(6.159951, 4.109598) = 29.406122$$

$$eval(U_8) = f(6.159951, 4.109598) = 29.406122$$

$$eval(U_9) = f(-2.687969, 5.361653) = 19.805119$$

$$eval(U_{10}) = f(0.474101, 4.170248) = 17.370896$$



2.1 遗传算法作业过程

至此，已完成遗传算法的第一代流程。依次迭代，在第451代得到对应最大目标函数值的染色体：

$$U_{best} = [111110000000111000 \quad 111101001010110]$$

相应实际值 $x_1=11.631407$, $x_2=5.724824$, 适应度值为：

$$eval(U) = f(11.631407, 5.724824) = 38.818208$$

结论：

(1) 遗传算法本质上是一种启发式的随机搜索算法，所以由遗传算法得出的结果每次都不尽相同。

(2) 自变量在给定的约束条件下进行了无缝编码（即这种编码方式能够表达解空间中的所以可行解），所以从理论上讲，遗传算法总有很多机会得到全局最优解而不是局部最优解。



2.1 启发式算法参数设计原则

在单纯的遗传算法中，也并不总是收敛，即使在单峰或单调也是如此。

原因：种群的进化能力已经基本丧失，种群早熟。

(1) **种群的规模**——规模小：近亲交配，病态基因，不能达到预期效果。规模大：结果难以收敛且浪费资源，稳健性下降。种群规模的一个建议值10~100.

(2) **交配概率**——概率小：不能有效更新种群。概率大：破坏已有的有利模式，随机性增大，错失最优个体。交配概率一般取0.4~0.9.

(3) **变异概率**——概率小：多样性下降太快。概率大：高阶模式被破坏的概率也随之增大。变异概率一般取0.0001~0.2.

(4) **进化代数**——代数小：算法不容易收敛，种群还没有成熟。代数大：算法已经成熟或早熟，继续进化没有意义。进化代数一般取100~500.

(5) **种群初始化**——初始种群的生成是随机的。但是最好尽量进行一个大概的区间估计，好处是：逼近最优解的编码空间，减轻算法负担。

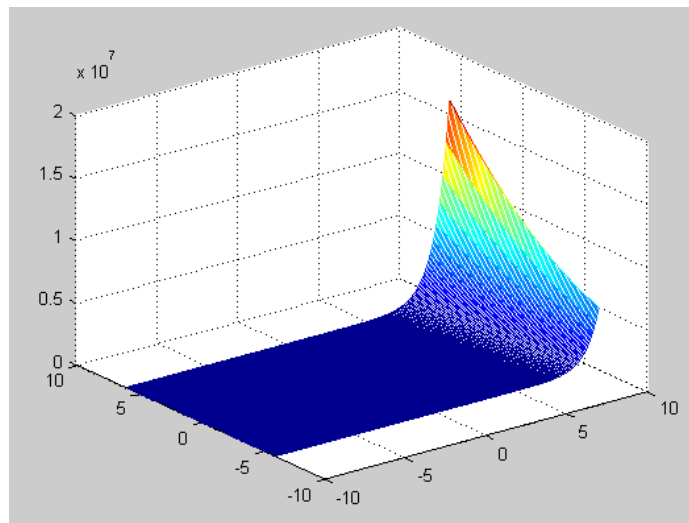


2.1 遗传算法案例

求解多约束非线性规划问题。

$$\begin{aligned} \min f(x) &= e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1) \\ \text{s.t.} \quad &\begin{cases} 1.5 + x_1x_2 - x_1 - x_2 \leq 0 \\ -x_1x_2 \leq 10 \end{cases} \end{aligned}$$

函数图像如下：





2.1 遗传算法工具箱的使用

(1) 编写M文件:

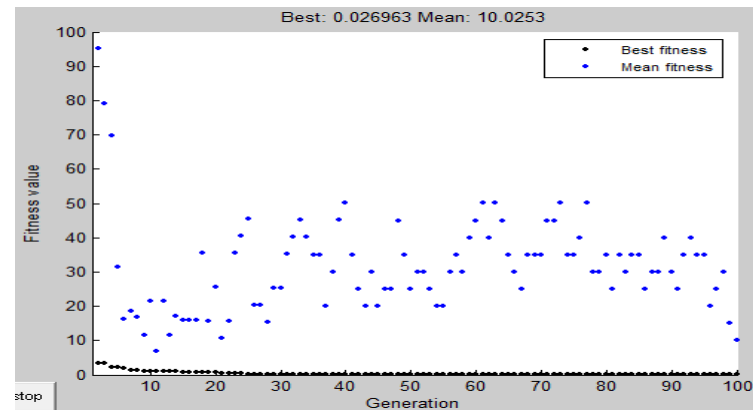
```
function f=youhua4(x)
g1=1.5+x(1)*x(2)-x(1)-x(2);g2=-x(1)*x(2)-10;
if (g1>0|g2>0)
    f=100;
else
    f=exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1);
end
```

(2) 命令窗口输入主程序:

```
T=100;
options=gaoptimset('Generations',T,'PlotFcns',@gaplotbestf);
[x,fval,reason,output,finnal_pop]=ga(@youhua4,2,options);
Bestx=x,BestFval=fval
```

(3) 结果:

```
Bestx =  -9.37208852995553  1.06025441141216
BestFval =  0.02696286452153
```



注意: 对于约束条件相对比较复杂, 导致可行解不太容易找到 (解空间对应区域的边界不规则), 直接用普通的交叉, 变异操作可能会很容易导致大量解不可行。解决办法是设计一个交叉、变异操作, 以便新生成的染色体尽量保持在可行解的区域内。



2.1 TSP问题-遗传算法求解

(旅行商问题) 某公司计划在SV地区做广告选宣传，推销员从城市1出发，经过各个乡镇，再回到城市1。为节约开支，公司希望推销员走过这10个城镇的总距离最少。

遗传算法思想求解TSP问题说明：

- (1) 思路和求最优值一样，
- (2) 开始是随机产生50条路径，
- (3) 路径的长度经过变化可作为适应度的值，并不需要转化成二进制进行交叉变异。
- (4) 能找到不同的路径，并给出答案。

新种群变异操作说明：

若路径为1：1→3→4→7→6→10→9→2→8→5→1

随机产生2个2~10之间不同的随机整数，变异后得新路径为：

新路径为：1→3→4→2→6→10→9→7→8→5→1

表 1 城镇之间的距离

	2	3	4	5	6	7	8	9	10
1	8	5	9	12	14	12	16	17	22
2		9	15	17	8	11	18	14	22
3			7	9	11	7	12	12	17
4				3	17	10	7	15	18
5					8	10	6	15	15
6						9	14	8	16
7							8	6	11
8								11	11
9									10



2.1 TSP问题-遗传算法求解

新种群交叉操作说明：

若路径为1：1→3→4→7→6→10→9→2→8→5→1 若路径为2：1→7→8→6→9→3→4→2→10→5→1

交叉后得新路径为：

新路径为1：1→3→4→7→9→3→4→2→8→5→1 新路径为2：1→7→8→6→6→10→9→2→10→5→1

经过扫描对比可得：

新路径为1：1→10→6→7→9→3→4→2→8→5→1 新路径为2：1→7→8→4→6→10→9→2→3→5→1

思考：还有其他交叉操作吗？

遗传算法求解结果： 经过1000代迭代，最佳路径为(所有路径)：

1	2	6	5	4	8	10	9	7	3	1
1	2	6	7	9	10	8	5	4	3	1
1	2	6	9	7	10	8	5	4	3	1

距离为：73， Elapsed time is 11.857000 seconds.



2.2 模拟退火算法

基本思想：

现代的模拟退火算法形成于20世纪80年代初，其思想源于固体的退火过程，即将固体加热至足够高的温度，再缓慢冷却；升温时，固体内部粒子随温度升高变为无序状，内能增大。而缓慢冷却时粒子又逐渐趋于有序，从理论上讲，如果冷却过程足够缓慢，那么冷却中任一温度时固体都能达到热平衡，而冷却到低温时将达到这一低温下的内能最小状态。



2.2 模拟退火算法

算法步骤:

- (1) 令 $T=T_0$ ，即开始退火的初始温度，随机生成一个初始解 x_0 ，并计算相应的目标函数值 $E(x_0)$ 。
- (2) 令 T 等于冷却进度表中的下一值 T_i 。
- (3) 根据当前解 x_i 进行扰动，产生一个新解 x_j ，计算相应的目标函数值 $E(x_j)$ ，得到 $\Delta E=E(x_j)-E(x_i)$ 。
- (4) 若 $\Delta E < 0$ ，则新解 x_j 被接受，作为新的当前解；
若 $\Delta E > 0$ ，则新解 x_j 按概率 $\exp(-\Delta E/T_i)$ 接受， T_i 为当前温度。
- (5) 在温度 T_i 下，重复 L_k 次的扰动和接受过程，即执行步骤(3)与(4)。
- (6) 判断 T 是否已达到 T_f ，是，则终止算法；
否，则转到步骤(2)继续执行。



2.2 模拟退火算法TSP程序与结果

经过迭代，最佳路径为：

1 3 7 9 10 8 4 5 6 2 1

距离为：73, Elapsed time is 3.058000 seconds.

```
clear
clc
tic
a=0.99; %温度衰减函数的参数
t0=97;tf=3;t=t0;
Length=100; %Markov链长度
amount=10; %城市数目
Dist=[0 8 5 9 12 14 12 16 17 22
      8 0 9 15 17 8 11 18 14 22
      5 9 0 7 9 11 7 12 12 17
      9 15 7 0 3 17 10 7 15 18
      12 17 9 3 0 8 10 6 15 15
      14 8 11 17 8 0 9 14 8 16
      12 11 7 10 10 9 0 8 6 11
      16 18 12 7 6 14 8 0 11 11
      17 14 12 15 15 8 6 11 0 10
      22 22 17 18 15 16 11 11 10 0]; %距离矩阵
```

```
sol_new=randperm(amount); %随机产生初始解
% sol_new是每次产生的新解; E_new是新解的回路距离
% sol_current是当前解; E_current是当前解对应的回路距离
% sol_best是冷却中的最好解; E_best是最优解
E_current=inf;
E_best=inf;
sol_current=sol_new;
sol_best=sol_new;
while t>=tf
    for r=1:Length
        %产生随机扰动
        if(rand<0.5) %随机决定是进行两交换还是三交换
            %两交换
            ind1=round(rand.*(amount-1))+1;
            ind2=round(rand.*(amount-1))+1;
            tmp1=sol_new(ind1);
            sol_new(ind1)=sol_new(ind2);
            sol_new(ind2)=tmp1;
        else
```

```
%三交换
            iind1=round(rand.*(amount-8))+1;
            iind2=round(rand.*(amount-9))+5;
            iind3=round(rand.*(amount-9))+7;
            tmp2=sol_new((iind1+1):(iind2-1));
            sol_new((iind1+1):(iind1+iind3-iind2+1))=sol_new(iind2:iind3);
            sol_new((iind1+iind3-iind2+2):iind3)=tmp2;
        end
        %计算目标函数值
        E_new=0;
        for i=1:(amount-1)
            E_new=E_new+Dist(sol_new(i),sol_new(i+1));
        end
        E_new=E_new+Dist(sol_new(amount),sol_new(1));
        if E_new<E_current
            E_current=E_new;
            sol_current=sol_new;
            if E_new<E_best
                %把冷却过程中最好的解保存下来
                E_best=E_new;
                sol_best=sol_new;
            end
        end
        %若新解的目标函数值小于当前解的，则仅以一定概率接受新解
        if rand<exp(-(E_new-E_current)./t)
            E_current=E_new;
            sol_current=sol_new;
        else
            E_new=E_current;
            sol_new=sol_current;
        end
    end
end
t=t.*a;
end
toc
disp('最优解为: ')
disp(sol_best)
disp('最短距离')
disp(E_best)
```



2.3 粒子群算法 (PSO算法)

粒子群算法的**思想**源于对鸟群捕食行为的研究。模拟鸟集群飞行觅食的行为，鸟之间通过集体的协作使群体达到最优目的，是一种基于Swarm Intelligence的优化方法。

- 每个寻优的问题解都被想像成一只鸟，称为“粒子”。所有粒子都在一个D维空间进行搜索。
- 所有的粒子都由一个 fitness function 确定适应值以判断目前的位置好坏。
- 每一个粒子必须赋予记忆功能，能记住所搜寻到的最佳位置。
- 每一个粒子还有一个速度以决定飞行的距离和方向。这个速度根据它本身的飞行经验以及同伴的飞行经验进行动态调整。



2.3 粒子群算法

D维空间中，有N个粒子；

粒子i位置： $x_i = (x_{i1}, x_{i2}, \dots, x_{iD})$ ，将 x_i 代入适应函数 $f(x_i)$ 求适应值；

粒子i速度： $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$

粒子i个体经历过的最好位置： $pbest_i = (p_{i1}, p_{i2}, \dots, p_{iD})$

种群所经历过的最好位置： $gbest = (g_1, g_2, \dots, g_D)$

通常，在第d ($1 \leq d \leq D$) 维的位置变化范围限定在 $[X_{\min,d}, X_{\max,d}]$ 内，

速度变化范围限定在 $[-V_{\max,d}, V_{\max,d}]$ 内（即在迭代中若 v_{id}, x_{id} 超出了边界值，则该维的速度或位置被限制为该维最大速度或边界位置）



2.3 粒子群算法

◎ 粒子i的第d维速度更新公式：

$$v_{id}^k = wv_{id}^{k-1} + c_1r_1(pbest_{id} - x_{id}^{k-1}) + c_2r_2(gbest_d - x_{id}^{k-1})$$

◎ 粒子i的第d维位置更新公式：

$$x_{id}^k = x_{id}^{k-1} + v_{id}^{k-1}$$

v_{id}^k — 第k次迭代粒子i飞行速度矢量的第d维分量

x_{id}^k — 第k次迭代粒子i位置矢量的第d维分量

c_1, c_2 — 加速度常数，调节学习最大步长

r_1, r_2 — 两个随机函数，取值范围[0, 1]，以增加搜索随机性

w — 惯性权重，非负数，调节对解空间的搜索范围

说明：粒子速度更新公式包含三部分：

第一部分为粒子先前的速度

第二部分为“**认知**”部分，表示粒子本身的思考，可理解为粒子i当前位置与自己最好位置之间的距离。

第三部分为“**社会**”部分，表示粒子间的信息共享与合作，可理解为粒子i当前位置与群体最好位置之间的距离。



2.3 粒子群算法流程

1. Initial:

初始化粒子群体（群体规模为 n ），包括随机位置和速度。

2. Evaluation:

根据fitness function，评价每个粒子的适应度。

3. Find the Pbest:

对每个粒子，将其当前适应值与其个体历史最佳位置（pbest）对应的适应值做比较，如果当前的适应值更高，则将用当前位置更新历史最佳位置pbest。

4. Find the Gbest:

对每个粒子，将其当前适应值与全局最佳位置（gbest）对应的适应值做比较，如果当前的适应值更高，则将用当前粒子的位置更新全局最佳位置gbest。

5. Update the Velocity:

根据公式更新每个粒子的速度与位置。

6. 如未满足结束条件，则返回步骤2

通常算法达到最大迭代次数或者最佳适应度值的增量小于某个给定的阈值时算法停止。



2.3 粒子群算法构成要素

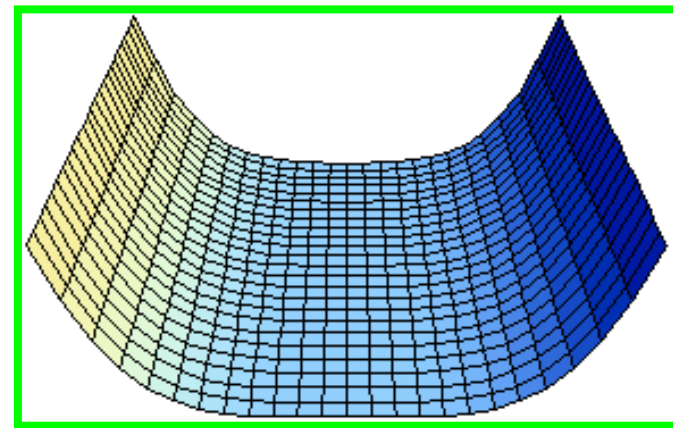
- 1. 群体大小：**是一个整型参数。m很小：陷入局优的可能性很大。m很大：PSO的优化能力很好，但收敛速度慢。当群体数目增长至一定水平时，再增长将不再有显著的作用。
- 2. 初始化：**较好地选择粒子的初始化空间，将大大缩短收敛时间。粒子群算法与其他现代优化方法相比的一个明显特色就是所需调整的参数很少。相对来说，**惯性因子**和**邻域定义**较为重要。这些为数不多的关键参数的设置却对算法的精度和效率有着显著影响。
- 3. 惯性因子：** $w=1$ 基本粒子群算法； $w=0$ 失去对粒子本身的速度的记忆。
- 4. 学习因子：** $c1=0$ 无私型粒子群算法，“只有社会，没有自我”迅速丧失群体多样性，易陷入局优而无法跳出。 $c2=0$ 自我认知型粒子群算法，“只有自我，没有社会”完全没有信息的社会共享，导致算法收敛速度缓慢。 $c1, c2$ 都不为0，称为完全型粒子群算法（保持收敛速度和搜索效果的均衡）。
- 5. 领域拓扑结构：**一种是将群体内所有个体都作为粒子的邻域，另一种是只将群体中的部分个体作为粒子的邻域。由此，将粒子群算法分为全局粒子群算法和局部粒子群算法。**全局：**粒子自己历史最优值、粒子群体的全局最优值。**局部：**粒子自己历史最优值、粒子邻域内粒子的最优值。
- 6. 停止准则：**最大迭代步数或可接受的满意解。



2.3 粒子群算法示例

求解如下四维Rosenbrock函数的优化问题.

$$\min f(\mathbf{x}) = \sum_{i=1}^3 [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$$
$$x_i \in [-30, 30] \ (i = 1, 2, 3, 4)$$



解析：算法的相关设计分析如下.

种群大小：即算法中粒子的数量，取 $m=5$

编码：因为问题的维数是4，所以每个粒子的位置和速度均4维的实数向量.

设定粒子的**最大速度** $V_{\max}=60$ ：

随机初始化各粒子的位置和速度.



2.3 粒子群算法示例

初始位置：

$$\begin{aligned}\mathbf{x}_1^{(0)} &= \{21.721, -9.13677, 6.62244, 3.84079\} \\ \mathbf{x}_2^{(0)} &= \{-13.5001, -23.6131, 17.4462, -29.0515\} \\ \mathbf{x}_3^{(0)} &= \{-29.6563, -0.871811, -27.8912, 17.7425\} \\ \mathbf{x}_4^{(0)} &= \{23.6218, -16.4885, -22.7019, 25.4033\} \\ \mathbf{x}_5^{(0)} &= \{-28.0992, 22.6482, 0.675616, -8.43752\}\end{aligned}$$

初始速度：

$$\begin{aligned}\mathbf{v}_1^{(0)} &= \{-19.9048, 29.562, -22.104, -5.45346\} \\ \mathbf{v}_2^{(0)} &= \{-20.5922, -28.6944, -26.3216, 19.0615\} \\ \mathbf{v}_3^{(0)} &= \{-7.83576, -55.7173, -40.9177, 28.255\} \\ \mathbf{v}_4^{(0)} &= \{-11.6373, -41.0138, 17.7311, -14.87\} \\ \mathbf{v}_5^{(0)} &= \{17.561, -13.5365, 51.2722, -56.098\}\end{aligned}$$

计算每个粒子的适应值

按照 $f(\mathbf{x}) = \sum_{i=1}^3 [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$ 计算适应值

群体历史最优解： $p_g = \mathbf{x}_1^{(0)}$

个体历史最优解： $p_i = \mathbf{x}_i^{(0)} (i = 1, 2, 3, 4, 5)$

$$f(\mathbf{x}_1^{(0)}) = 2.38817 \times 10^7$$

$$f(\mathbf{x}_2^{(0)}) = 4.45306 \times 10^7$$

$$f(\mathbf{x}_3^{(0)}) = 1.35376 \times 10^8$$

$$f(\mathbf{x}_4^{(0)}) = 6.56888 \times 10^7$$

$$f(\mathbf{x}_5^{(0)}) = 8.50674 \times 10^7$$



2.3 粒子群算法示例

取 $w=1, c1=c2=1$, 得到速度和位置的更新函数为

$$\mathbf{v}_{k+1} = \mathbf{v}_k + 2r_1(\mathbf{p}_k - \mathbf{x}_k) + 2r_2(\mathbf{p}_g - \mathbf{x}_k),$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_{k+1}$$

更新速度, 得:

$$\begin{aligned}\mathbf{v}_1^{(1)} &= \{-19.9048, 29.562, -22.104, -5.45346\} \\ \mathbf{v}_2^{(1)} &= \{40.0498, -3.76972, -44.9573, 75.6939(60)\} \\ \mathbf{v}_3^{(1)} &= \{14.8665, -59.3694, -25.667, 22.1122\} \\ \mathbf{v}_4^{(1)} &= \{-13.843, -32.4824, 51.7604, -39.892\} \\ \mathbf{v}_5^{(1)} &= \{95.9018(60), -63.5174(-60), 60.6234(60), -36.7907\}\end{aligned}$$

更新位置, 得:

$$\begin{aligned}\mathbf{x}_1^{(1)} &= \{1.81621, 20.4252, -15.4816, -1.61267\} \\ \mathbf{x}_2^{(1)} &= \{26.5497, -27.3829, -27.5112, 30.9485\} \\ \mathbf{x}_3^{(1)} &= \{-14.7898, -60.2412, -53.5582, 39.8547\} \\ \mathbf{x}_4^{(1)} &= \{9.77877, -48.971, 29.0584, -14.4887\} \\ \mathbf{x}_5^{(1)} &= \{31.9008, -37.3518, 60.6756, -45.2282\}\end{aligned}$$

位置一般不强
行拉回解空间

按公式计算适应值:

目前历史最优解

$$f(\mathbf{x}_1^{(1)}) = 2.35726 \times 10^7$$

$$f(\mathbf{x}_2^{(1)}) = 1.6674 \times 10^8$$

$$f(\mathbf{x}_3^{(1)}) = 2.16403 \times 10^9$$

$$f(\mathbf{x}_4^{(1)}) = 6.37125 \times 10^8$$

$$f(\mathbf{x}_5^{(1)}) = 1.6783 \times 10^9$$

继续找出个体最优解, 重复上述步骤, 将迭代进行下去. 经过10000次迭代, 粒子群算法得到了比较好的适应值。



3. 机器学习算法概述

机器学习 (Machine Learning, ML) 是一门多领域交叉学科，涉及概率论、统计学、逼近论、凸分析、算法复杂度理论等多门学科。专门研究计算机怎样**模拟或实现人类的学习行为**，以获取新的知识或技能，重新组织已有的知识结构使之不断改善自身的性能。

机器学习是**人工智能**的核心，是使计算机具有智能的根本途径，其应用遍及人工智能的各个领域。例如：数据挖掘、计算机视觉、自然语言处理、生物特征识别、搜索引擎、医学诊断、DNA序列测序、语音和手写识别、战略游戏和机器人运用等。

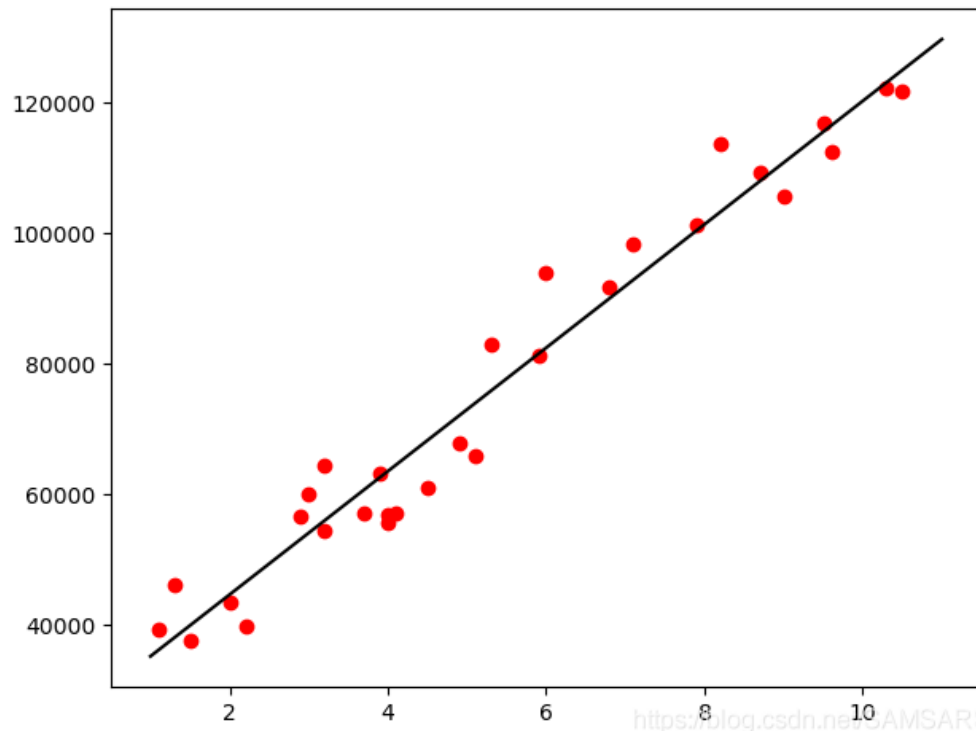
机器的能力能否超过人？值得探讨。



3.1 机器学习算法之线性回归

线性回归 (Linear Regression) 可能是最流行的机器学习算法。线性回归就是要找一条直线，并且让这条直线尽可能地拟合散点图中的数据点。它试图通过将直线方程与该数据拟合来表示自变量 (x 值) 和数值结果 (y 值)。然后就可以用这条线来预测未来的值！

这种算法最常用的技术是最小二乘法 (Least of squares)。这个方法计算出最佳拟合线，以使得与直线上每个数据点的垂直距离最小。总距离是所有数据点的垂直距离的平方和。其思想是通过最小化这个平方误差或距离来拟合模型。

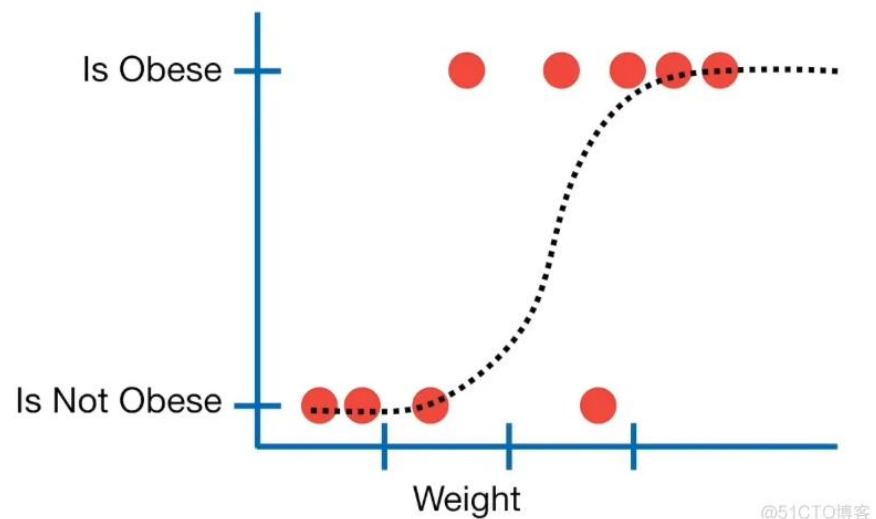




3.2 机器学习算法之逻辑回归

逻辑回归 (Logistic regression) 与线性回归类似，但它是用于输出为二进制 (多进制也有) 的情况 (即，当结果只能有两个可能的值)。一般用于需要明确输出的场景，如某些事情的发生 (预测是否会发生降雨)。通常，逻辑回归使用某种函数将概率值压缩到某一特定范围。对最终输出的预测是第一个非线性的 S 型函数，称为 logistic function。

这个逻辑函数将中间结果值映射到结果变量 Y ，其值范围从 0 到 1。然后，这些值可以解释为 Y 出现的概率。S 型逻辑函数的性质使得逻辑回归更适合用于分类任务。

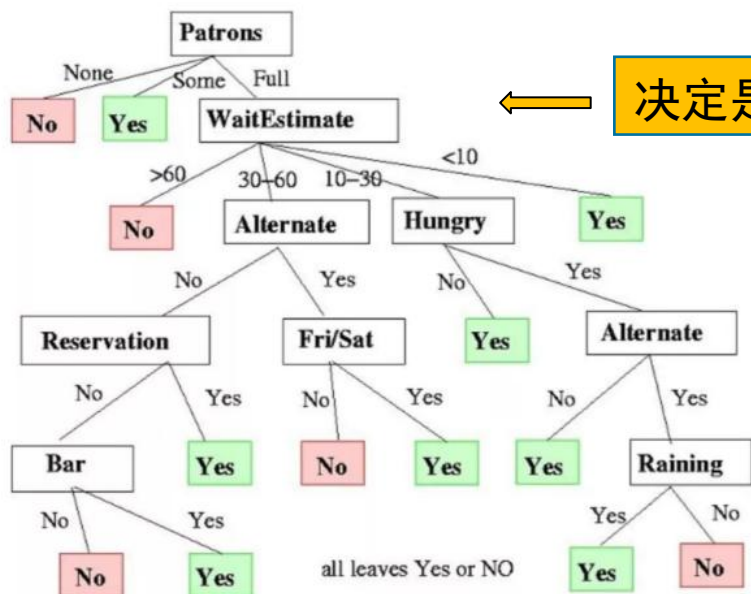




3.3 机器学习算法之决策树

如果说线性和逻辑回归都是把任务在一个回合内结束，那么**决策树**（Decision Trees）就是一个多步走的动作，它同样用于**回归和分类**任务中，不过场景通常更复杂且具体。

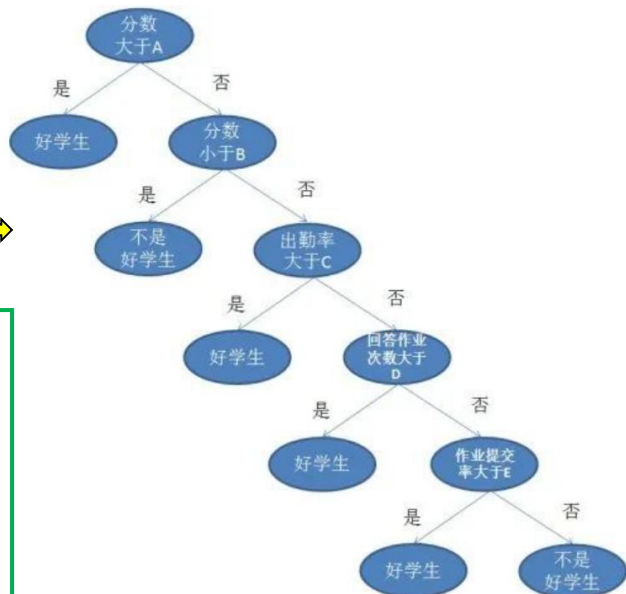
在这一算法中，训练模型通过**学习树**表示（Tree representation）的决策规则来学习预测目标变量的值。树是由具有相应属性的节点组成的。在每个节点上，我们根据可用的特征询问有关数据的问题。左右分支代表可能的答案。最终节点（即叶节点）对应于一个预测值。每个特征的重要性是通过自顶向下方法确定的。**节点越高，其属性就越重要。**



决定是否在餐厅等的决策树示例

是否为好学生的决策树示例

老师面对一个班级的学生，哪些是好学生？如果简单判断考试90分就算好学生好像太粗暴了，不能唯分数论。那面对成绩不到90分的学生，我们可以从作业、出勤、提问等几个方面分开讨论。



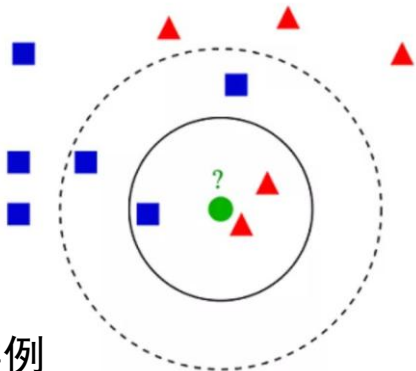


3.4 机器学习算法之最近邻算法 (KNN)

K- 最近邻算法 (K-Nearest Neighbors, KNN) 非常简单。KNN 通过在整个训练集中搜索 K 个最相似的实例，即 K 个邻居，并为所有这些 K 个实例分配一个公共输出变量，来对对象进行**分类**。**三个基本要素：K值选择、距离度量、分类决策规则。**

K 的选择很关键：较小的值可能会得到大量的噪声和不准确的结果，而较大的值是可行的。它最常用于分类，但也适用于回归问题。

用于评估实例之间相似性的距离可以是**欧几里得距离** (Euclidean distance)、**曼哈顿距离** (Manhattan distance) 或**明氏距离** (Minkowski distance) 等。欧几里得距离是两点之间的普通直线距离。它实际上是点坐标之差平方和的平方根。



▲ KNN 分类例

训练集有两类（蓝色、红色），新来了一个样本（绿色），要判断其属于哪一类。KNN算法就是在整个特征空间找到最近的K个参考对象（K=3），这K个对象里属于哪一个类别的数目多，就是赋予新样本对应的标签。比如这个例子中新样本就为**红色**类别。

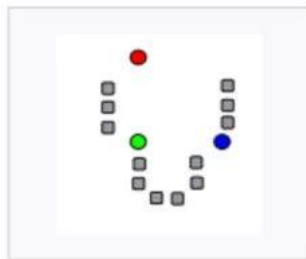


3.5 机器学习算法之K-均值

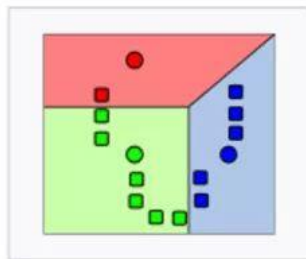
K- 均值 (K-means) 是通过对数据集进行分类来**聚类**的。例如，这个算法可用于根据购买历史将用户分组。它在数据集中找到 K 个聚类。K-均值用于无监督学习，因此，我们只需使用训练数据 X ，以及我们想要识别的聚类数量 K 。

该算法根据每个数据点的特征，将每个数据点**迭代**地分配给 K 个组中的一个组。它为每个 K - 聚类（称为质心）选择 K 个点。基于相似度，将新的数据点添加到具有最近质心的聚类中。这个过程一直持续到**质心停止变化**为止。

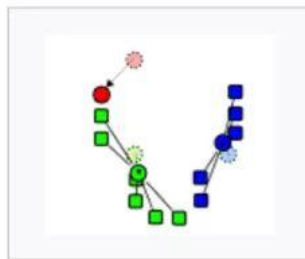
生活中，K-均值在欺诈检测中扮演了重要角色，在汽车、医疗保险和保险欺诈检测领域中广泛应用。



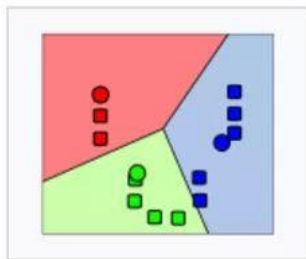
1. k initial "means" (in this case $k=3$) are randomly generated within the data domain (shown in color).



2. k clusters are created by associating every observation with the nearest mean. The partitions here represent the Voronoi diagram generated by the means.



3. The centroid of each of the k clusters becomes the new mean.



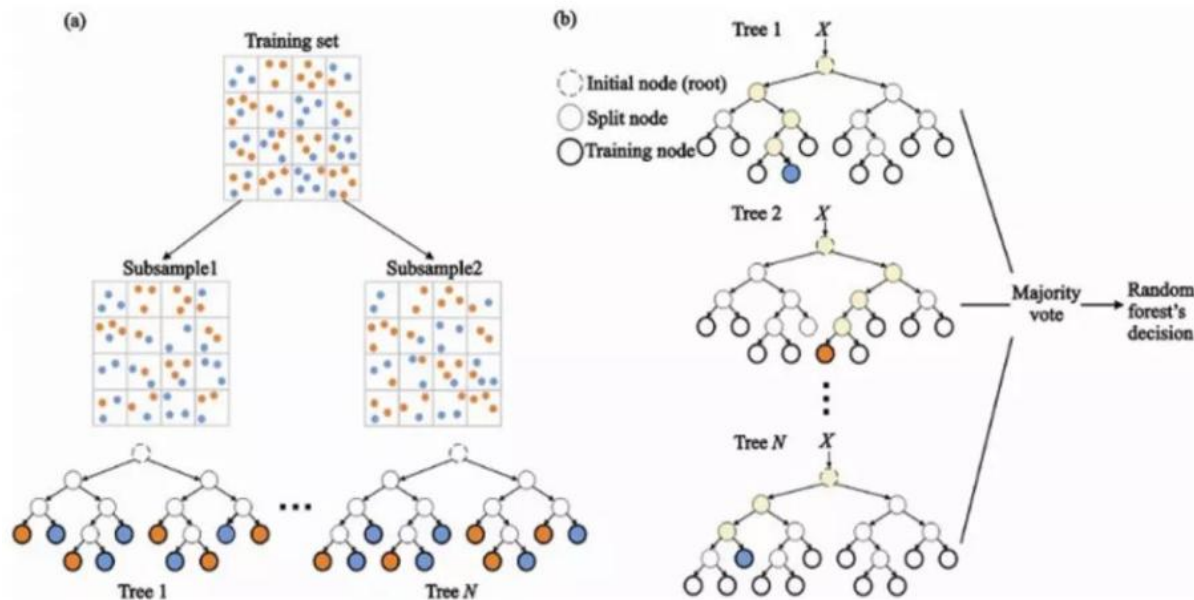
4. Steps 2 and 3 are repeated until convergence has been reached.



3.6 机器学习算法之随机森林

随机森林 (Random Forest) 是一种非常流行的集成机器学习算法。这个算法的基本思想是，**许多人的意见要比个人的意见更准确**。在随机森林中，我们使用决策树集成（参见决策树）。为了对新对象进行**分类**，我们从每个决策树中进行投票，并结合结果，然后根据多数投票做出最终决定。

- (a) 在训练过程中，每个决策树都是基于训练集的引导样本来构建的。
- (b) 在分类过程中，输入例的决定是根据多数投票做出的。



解释：从原始的数据集中采取有放回的抽样，构造子数据集，子数据集的数据量是和原始数据集相同的。不同子数据集的元素可以重复，同一个子数据集中的元素也可以重复。

利用子数据集来构建子决策树，将这个数据放到每个子决策树中，每个子决策树输出一个结果。最后，如果有了新的数据需要通过随机森林得到分类结果，就可以通过对子决策树的判断结果的投票，得到随机森林的输出结果了。如图所示，随机森林中有N棵子决策树，统计每个树的结果，如果蓝色多于红色，那么随机森林的分类结果就是蓝色结果。

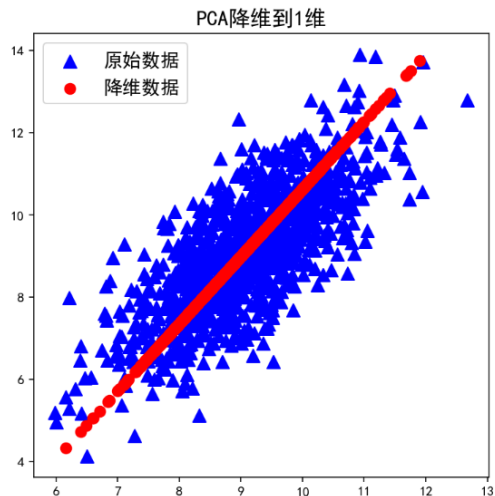


3.7 机器学习算法之降维

由于我们今天能够捕获的数据量之大，机器学习问题变得更加复杂。这就意味着训练极其缓慢，而且很难找到一个好的解决方案。这一问题，通常被称为“维数灾难”（Curse of dimensionality）。

降维（Dimensionality reduction）试图在不丢失最重要信息的情况下，通过将特定的特征组合成更高层次的特征来解决这个问题。

主成分分析（Principal Component Analysis, PCA）是最流行的降维技术。主成分分析通过将数据集压缩到低维线或超平面 / 子空间来降低数据集的维数。这尽可能地保留了原始数据的显著特征。



可以通过将所有数据点近似到条直线来实现降维的示例。

降维方法：线性和非线性降维，非线性降维又分为基于核函数和基于特征值的方法。

1、线性降维方法：PCA、ICA、LDA、LFA、LPP。

2、非线性降维方法：

(1) 基于核函数的方法：KPCA、KICA、KDA

(2) 基于特征值的方法（流型学习）：

ISOMAP、LLE、LE、LPP、LTSA、MVU

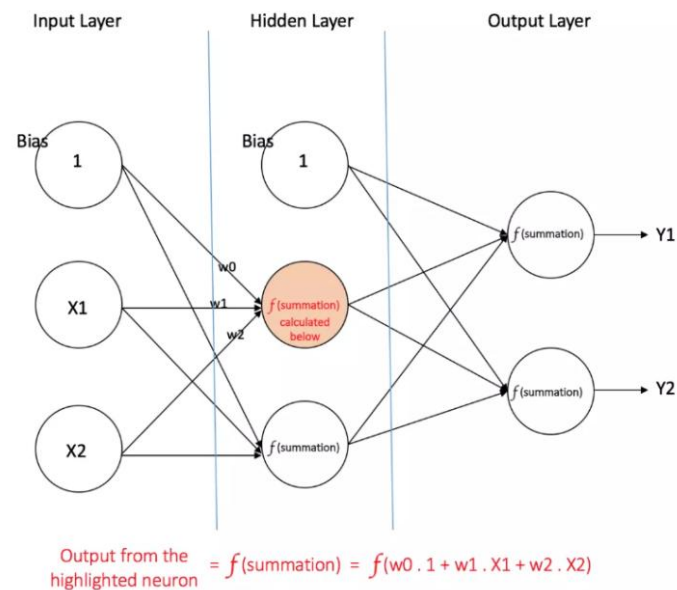


3.8 机器学习算法之人工神经网络 (ANN)

人工神经网络 (Artificial Neural Networks, ANN) 可以处理大型复杂的机器学习任务。神经网络本质上是一组带有权值的边和节点组成的相互连接的层，称为**神经元**。在输入层和输出层之间，我们可以插入多个**隐藏层**。人工神经网络使用了两个隐藏层。除此之外，还需要处理深度学习。

人工神经网络的工作原理与大脑的结构类似。一组神经元被赋予一个随机权重，以确定神经元如何处理输入数据。通过对输入数据训练神经网络来学习输入和输出之间的关系。在训练阶段，系统可以访问正确的答案。如果网络不能准确识别输入，系统就会调整权重。经过充分的训练后，它将始终如一地识别出正确的模式。

每个圆形节点表示一个人工神经元，箭头表示从一个人工神经元的输出到另一个人工神经元的输入的连接。





3.9 机器学习教程推荐



第一篇 监督学习

第1章 统计学习及监督学习概论

第2章 感知机

第3章 K近邻法

第4章 朴素贝叶斯法

第5章 决策树

第6章 逻辑斯蒂回归与最大熵模型

第7章 支持向量机

第8章 提升方法

第9章 EM算法及推广

第10章 隐马尔可夫模型

第11章 条件随机场

第12章 监督学习方法总结

第二篇 无监督学习

第13章 无监督学习概论

第14章 聚类方法

第15章 奇异值分解

第16章 主成分分析

第17章 潜在语义分析

第18章 概率潜在语义分析

第19章 马尔可夫链蒙特卡罗法

第20章 潜在狄利克雷分配

第21章 PageRank算法

第22章 无监督学习方法总结



3.9 机器学习教程推荐



《机器学习方法》在《统计学习方法》的基础上增加了深度学习的内容，全面系统地介绍了机器学习的主要方法，系统阐述其理论、模型、策略和算法，从具体例子入手，由浅入深，帮助读者直观地理解基本思路，同时从理论角度出发，给出严格的数学推导，严谨详实，让读者更好地掌握基本原理和概念。

本书的定位是讲解机器学习的基本内容，但不完全是入门书。主旨是把最重要的原理和方法做系统的总结，方便大家经常阅读和复习。

第三篇 深度学习

第23章 前馈神经网络

第24章 卷积神经网络

第25章 循环神经网络

第26章 序列到序列模型

第27章 预训练语言模型

第28章 生成对抗网络

第29章 深度学习方法总结



4.1 TSP问题-改进遗传算法求解

（**旅行商问题**）某公司计划在SV地区做广告选宣传，推销员从城市1出发，经过各个乡镇，再回到城市1。为节约开支，公司希望推销员走过这**50个城镇**的总距离最少。

遗传算法思想求解TSP问题说明：

- （1）思路和求最优值一样（采用十进制编码）。
- （2）随机产生50条路径（建议采用改良圈算法）并计算距离，将每条路径的距离转化为适应度的值。

$$v_i = \frac{\sum_{j=1}^{50} d_j}{d_i}, i = 1, 2, \dots, 50$$

- （3）根据适应度的值计算选择概率与累计概率，随机选择两条路径出来。
- （4）对选择出来的两条路径进行交配操作（单点交叉\多点交叉\部分匹配交叉\顺序交叉等）。
- （5）在交配完成后，在进行独立的变异操作。（本题：交配概率1（充分进化），变异概率0.1）
- （6）重复上述过程，得到新的50条路径（子代），保留好此时的最佳路径与距离。
- （7）迭代200-1000次，给出最短路径的距离和方案。



4.1 编码、交叉和变异操作

1. 编码策略（不唯一）

采用十进制编码，用随机数列 $w_1w_2\cdots w_{51}$ 作为染色体，其中 $0\leq w_i\leq 1$ ($i=2, 3, \cdots, 50$)， $w_1=0$ （起点）， $w_{51}=1$ （终点，起点与终点相同）；每一个随机序列都和种群中的一个个体相对应。

例如：9目标问题的一个染色体为

[0. 23, 0. 82, 0. 45, 0. 74, 0. 87, 0. 11, 0. 56, 0. 69, 0. 78]

则： $w_1=0. 23, w_2=0. 82, \cdots, w_9=0. 78$ ($w_6<w_1<w_3<w_7<w_8<w_4<w_9<w_2<w_5$)

式中：位置 i 的机数表示目标 i 在巡回中的顺序。

将这些随机数按照升序排列得到如下巡回：

6 → 1 → 3 → 7 → 8 → 4 → 9 → 2 → 5



4.1 编码、交叉和变异操作

2. 交叉操作（不唯一）

对于选定的两个父代个体，随机地选取第 t 个基因处为交叉点，从 $t+1$ 到结束进行交换。

例如： $t=4$ ，则

原始： $f1 = [0, 0.14, 0.25, 0.27, 0.29, 0.54, \dots, 0.19, 1]$

$f2 = [0, 0.23, 0.44, 0.56, 0.74, 0.21, \dots, 0.24, 1]$

交叉后： $s1 = [0, 0.14, 0.25, 0.27, 0.74, 0.21, \dots, 0.24, 1]$

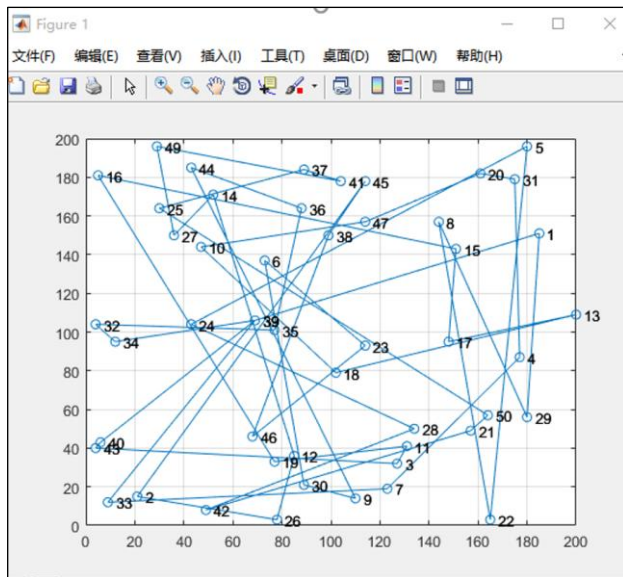
$s2 = [0, 0.23, 0.44, 0.56, 0.29, 0.54, \dots, 0.19, 1]$

3. 变异操作（不唯一）

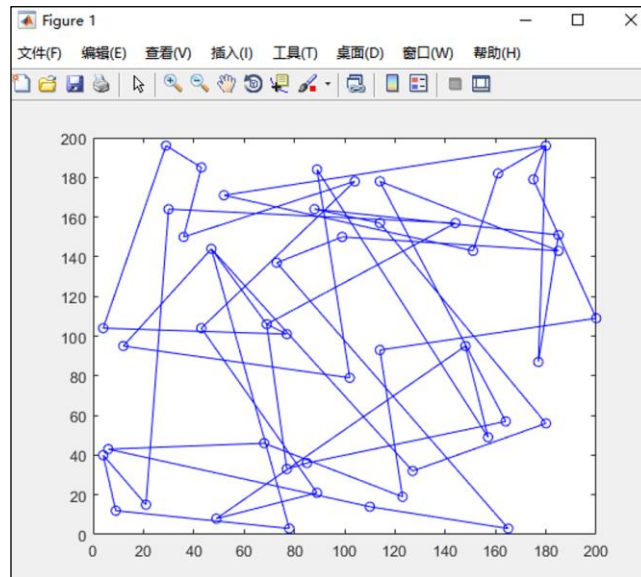
按照给定的变异率，随机地选取三个整数，满足 $1 < u < v < w < 51$ ，把 u, v 之间（包括 u, v ）的基因段插到 w 后面。



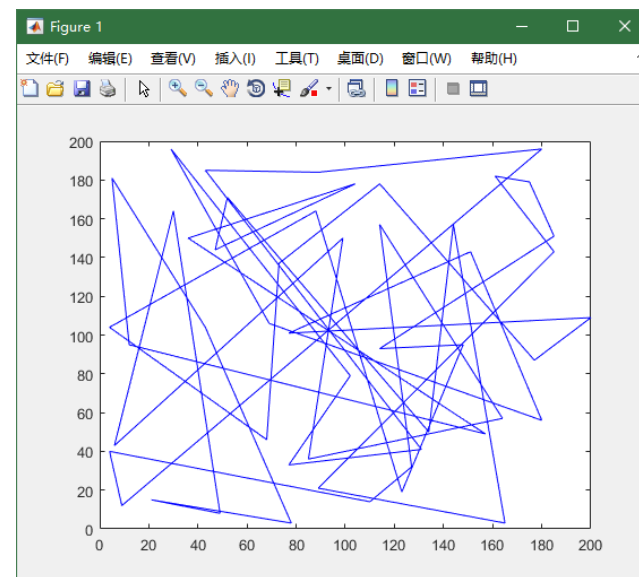
4.1 结果展示



(同学1：路程4257.87km)



(同学2：路程3900.84km)



(同学3：路程4753.20km)

结果说明： 同学们的结果不太理想，问题出现在什么地方？

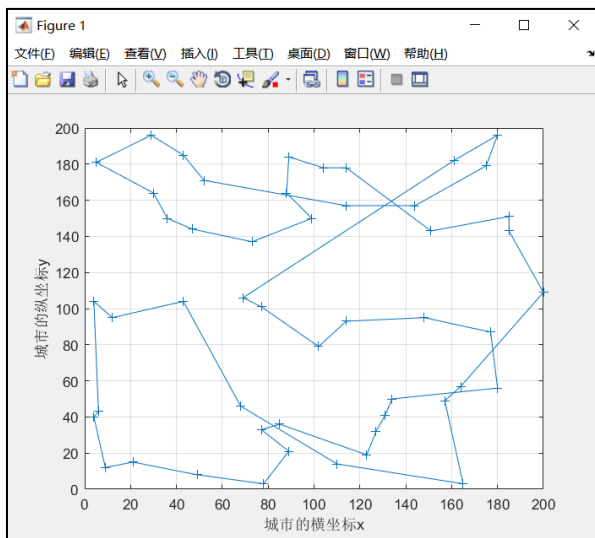
1. 适应度函数 （好的路线与差的路线区分度不够，导致迭代过程中无法区别）
2. 交叉操作 （随机交叉波动性太大→改进型交叉操作）
3. 变异操作 （单点变异→多点变异）



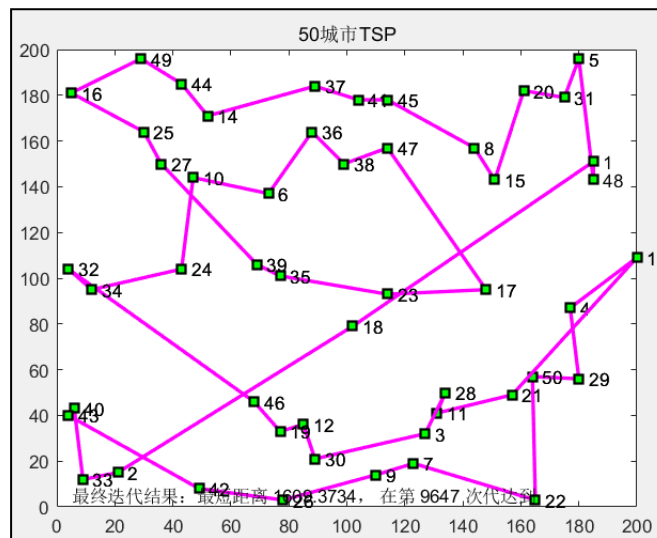
4.1 改进的遗传算法-1

1. 适应度函数（非唯一，增加区分度）

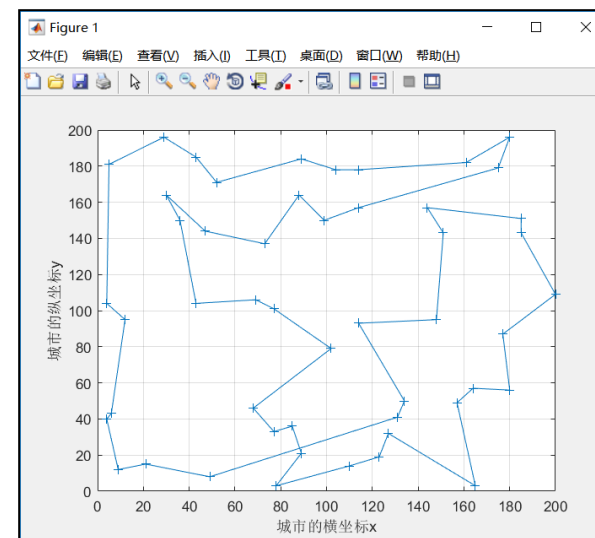
$$v_i = \frac{\sum_{j=1}^{50} d_j}{d_i}, i = 1, 2, \dots, 50 \Rightarrow v_i = \left[\frac{\sum_{j=1}^{50} d_j}{d_i} \right]^3, i = 1, 2, \dots, 50$$



(同学4: 路程1488.67m)



(同学5: 路程1609.37m)



(同学6: 路程1428.68m)



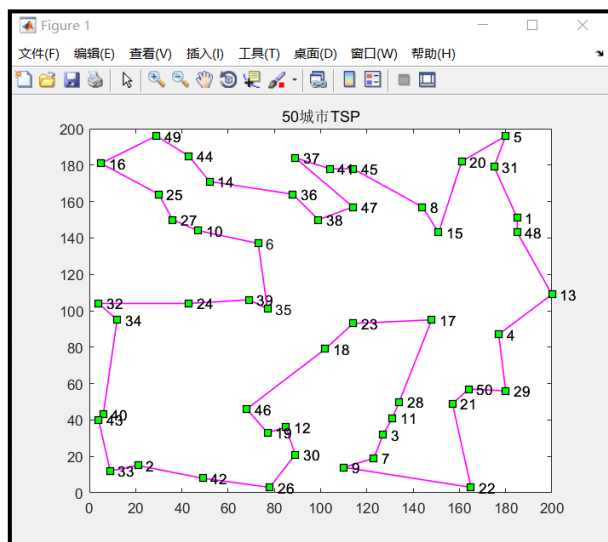
4.1 改进的遗传算法-2

2. 改进型交叉

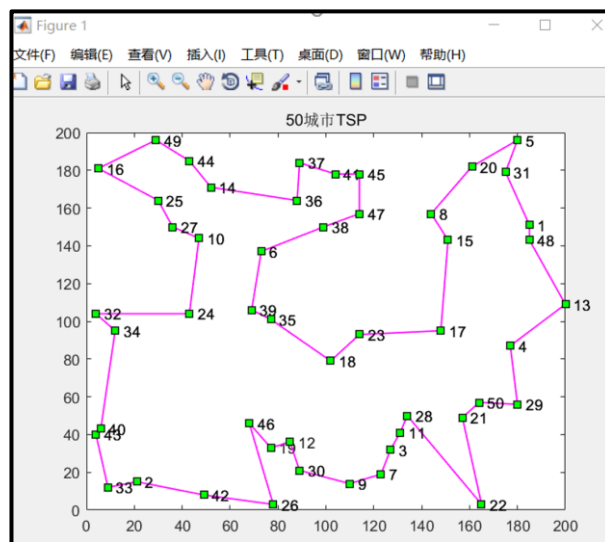
① 以“门当户对”原则，对父代个体进行配对，即对父代以适应度函数值进行排序，值小的与小的配对，值大的与大的配对。

② 利用混沌序列确定交叉点的位置，然后对确定的交叉项进行交叉（单点交叉）。

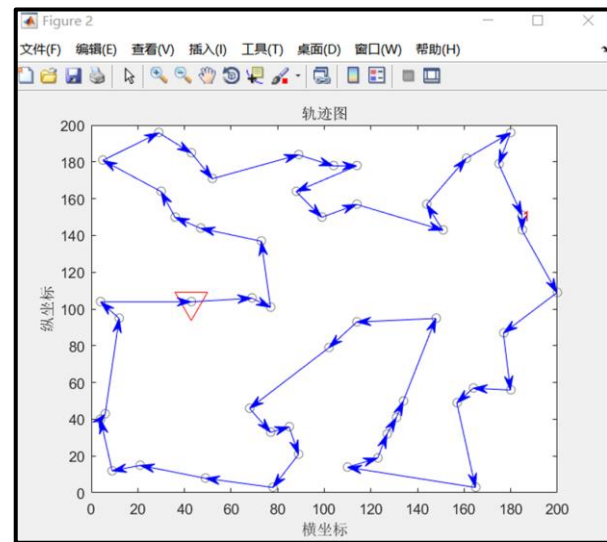
譬如：Logistic混沌序列 $x(n+1)=4x(n)[1-x(n)]$ 产生一个2~50之间的正整数。



(同学7：路程1215.49m)



(同学8：路程1198.85m)



(同学9：路程1203.67m)



5.1 复杂问题处理技巧之多目标问题常用解法

一、传统经典方法：这类方法的核心是将多目标问题转化为单目标问题或一系列单目标问题。

1. 加权求和法

- 原理：对每个目标函数赋予权重，将多目标问题转化为“加权和单目标函数”的优化问题。

$$\min F(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_n f_n(x) \quad \text{其中：权重 } w_i \geq 0, \sum_{i=1}^n w_i = 1$$

- 优点：简单直观，计算效率高，可直接利用单目标优化算法。
- 缺点：权重难以客观确定；仅能求解凸帕累托前沿，非凸前沿可能丢失部分解；对权重敏感（微小变化可能导致解大幅波动）。
- 适用场景：目标函数为凸函数，且决策者可明确权重偏好。

2. ϵ 约束法：选择一个目标作为主要优化目标，将其他目标转化为约束条件（设定阈值）。

3. 目标规划法：为每个目标设定一个期望达到的目标值（或理想点）。然后最小化所有目标与各自目标值的偏差（通常是绝对偏差或平方偏差）的加权和。

4. 理想点法：先求解每个目标的单目标最优解，再最小化实际解与理想点的距离（如欧氏距离、切比雪夫距离）。



5.1 复杂问题处理技巧之多目标问题常用解法

二、启发式/智能优化算法（基于种群搜索）：对于复杂多目标问题（如非线性、非凸、高维、离散变量），传统方法难以求解，需依赖启发式算法。这类方法通过种群进化直接搜索帕累托前沿，无需数学转化

1. 多目标遗传算法（MOGA）及改进算法：NSGA-II（Non-dominated Sorting Genetic Algorithm II）、NSGA-III.

- 原理：基于遗传算法框架，通过**非支配排序**（将种群分为不同层级的帕累托解）和拥挤度计算（保持解的多样性，避免聚集）选择优秀个体，实现帕累托前沿的逼近。
- 优点：可处理复杂非线性问题，能同时生成多个帕累托解，鲁棒性强。
- 缺点：计算量大，需调参（如种群规模、交叉 / 变异概率）；对高维目标（ $k > 5$ ）性能下降。
- 适用场景：工程优化、机器学习超参数调优、路径规划等复杂问题。

2、多目标分解算法（MOEA/D）：将多目标问题分解为 N 个单目标子问题（如使用加权和法、Tchebycheff法或边界交叉法）同时优化这些子问题。每个子问题关联一个权重向量（定义方向），每个子问题对应一个帕累托解，利用子问题间的信息共享（如邻域协作）同时优化所有子问题。

3、多目标粒子群优化（MOPSO）、多目标蚁群优化（MOACO）、多目标模拟退火（MOSA）等。



5.1 复杂问题处理技巧之多目标问题常用解法

三、交互式方法（结合决策者偏好）：多目标问题的最终解需由决策者根据偏好选择，交互式方法通过算法与决策者的迭代反馈逐步逼近满意解。

1. STEM 方法（Step Method）

- 原理：先求解初始帕累托解，展示各目标值；决策者对不满意的目標提出改进要求，算法通过约束调整生成新解，反复迭代直至决策者满意。
- 优缺点：无需预先设定权重，动态融入决策者偏好。依赖决策者参与，迭代次数可能较多。

2. 偏好锥方法（Preference Cone Method）：决策者通过定义“偏好锥”（如“更关注目标 1 和目标 2 的改进”），算法在锥内搜索帕累托解，缩小解空间。

总结：简单问题（线性 / 凸函数）：优先选择加权求和法、约束法。

复杂问题（非线性 / 非凸 / 高维）：采用 NSGA-II、MOEA/D 等智能优化算法。

需决策者参与：使用交互式方法（如 STEM）或结合启发式算法的偏好引导搜索。

混合方法：如“加权法 + NSGA-II”（先用传统方法缩小范围，再用启发式搜索）。

自适应方法：动态调整算法参数（如 NSGA-II 的交叉概率）或目标权重，提升收敛效率。



5.1 复杂问题处理技巧之多目标问题常用解法 (2020A)

★ 2020国赛A题第四问（双目标处理）——全国专家建议方法

目标直接相加（不行） $\min S + R$

处理后的目标：先按照单目标求出每个目标的最大或最小值，再按照如下其中一个公式计算结果（思考：哪些方式更合理？）。

$$\min \frac{S}{S_{\max}} + \frac{R}{R_{\max}}$$

$$\max \frac{S_{\max}}{S} + \frac{R_{\max}}{R}$$

$$\max \frac{S_{\min}}{S} + \frac{R_{\min}}{R}$$

$$\min \frac{S}{S_{\min}} + \frac{R}{R_{\min}}$$

$$\min \frac{S - S_{\min}}{S_{\max} - S_{\min}} + \frac{R - R_{\min}}{R_{\max} - R_{\min}}$$



5.2 复杂问题处理技巧之**粗细网格遍历** (2020A)

原因：实际上, 问题 3 和问题 4 的求解困难很大程度上是制程界限 H 条件带来的——为了判断是否满足 制程界限 H , 只能将炉温曲线以某种方式求解出来, 这就涉及到偏微分方程的求解. 而由于边界条件和方程中的系数是分段线性的, 解析解很困难, 在温度与速度的设置下解析求解最优曲线同样困难.

因此,可行的办法是: 以某种方式 “遍历” 所有的温度与速度的设置, 计算并挑选出符合制程界限的所有炉温曲线即曲线族 u , 在 u 中计算出最优曲线. 当然**均匀采样**会是首选, 但这会使得采样点的个数随着空间的**维数指数增长**.

“粗细网格遍历” 就是温度和速度在 “大步长下” 遍历全部设置, 根据计算结果在大步长下 “最优曲线” 的温度和速度设置附近, 加细步长, 以找到最优曲线.



5.3 复杂问题处理技巧之混沌迭代(近似遍历)(2020A)

通过“混沌迭代”进行“混沌采样”的办法,实现近似的“遍历”。给定初值 $x_1 \in (0,1)$ 后可以生成 $(0,1)$ 之间的一个混沌序列 $\{x_k\}_{k=1}^M : x_{k+1} = x_k(1-x_k)$. 在得到一个长度为 M 的混沌序列 $\{x_k\}_{k=1}^M$ 后,经过简单的线性变换后可将其映射到区间 (a,b) 的一个混沌序列 $\{y_k\}_{k=1}^M$, 其中, $y_k = (b-a)x_k + a$. 基于此,可以生成一种有 M 种参数设置的“混沌采样”结果,从而在有限的时间内求解出规模较大的问题。关于 M 的设置, M 越大结果越好,但计算量也会增大。以计算问题 3 的最优曲线为例,给出如下解决方法:

- ① 给定 $M_0 = m_0$, 计算出最优曲线对应的面积 S_0 ;
- ② 令 $M_1 = M_0 + d$, 计算出最优曲线对应的面积 S_1 , 并计算 S_0 和 S_1 的相对误差 $|S_0 - S_1|/S_1$;
- ③ 若相对误差小于 5%, 则认为 ② 中面积为 S_1 , 对应的曲线即为最优曲线;
- ④ 若相对误差大于 5%, 则令 $M_0 = M_1$, 然后重复 ①、②直至 ③ 成立。



5.4 复杂问题处理技巧之混合算法

有时候结合使用多种优化方法或技术可以获得更好的结果。例如，可以先使用启发式算法获得一个初始解，然后再使用精确算法进行进一步优化。

方法：“遗传算法（启发式）+ 2-opt 局部搜索（精确优化）” 求解车辆路径问题（VRP）

背景：某连锁超市的区域物流中心需为 15 个社区客户配送货物，具体约束和目标如下：

- 物流中心（编号 0）为起点和终点；
- 15 个客户（编号 1-15），每个客户有固定需求量（10-30kg）；
- 可用配送车辆 3 辆，每辆车最大载重 80kg；
- 目标：在满足载重约束的前提下，最小化总行驶距离（单位：km）。

混合优化步骤：

第一步：用遗传算法生成初始解，经过50次迭代，生成的初始解如下：

- 车辆1路径：0 → 1 → 4 → 7 → 10 → 0，路程28km，总需求75kg
- 车辆2路径：0 → 2 → 5 → 9 → 12 → 15 → 0，路程42km，总需求78kg
- 车辆3路径：0 → 3 → 6 → 8 → 11 → 13 → 14 → 0，路程45km，总需求72kg
- 初始解总距离：28+42+45=115km



5.4 复杂问题处理技巧之混合算法

第二步：用 2-opt 局部搜索（精确优化）细化解

遗传算法生成的初始解可能存在路径交叉或冗余，需用局部精确算法进一步优化。**2-opt 算法**通过消除路径中的交叉和迂回，精细降低距离。

- 车辆 2 路径优化后结果：0 → 2 → 9 → 5 → 12 → 15 → 0，路程38km（降低4km）
- 车辆 3 路径优化后结果：0 → 3 → 6 → 11 → 8 → 13 → 14 → 0，路程40km（降低5km）
- 优化后总距离：28+38+40=106km

效果对比与原理分析

阶段	总行驶距离	计算耗时（普通电脑）	解的质量
遗传算法初始解	115km	约 2 分钟（50 代迭代）	较优但存在冗余
2-opt 优化后	106km	额外 30 秒（局部搜索）	更接近最优解

为什么混合策略有效？（兼顾**计算精度**和**计算效率**）

1.遗传算法的“探索”作用：快速在 15 个客户的复杂解空间（总可能路径数超 10^{12} ）中定位到“可行且较优”的初始解，避免了盲目搜索；

2.2-opt 的“求精”作用：针对初始解中局部路径的冗余（交叉、迂回）进行精确调整，利用数学规律（两点间直线最短）消除浪费，弥补了启发式算法可能忽略的细节；

3.效率平衡：若直接用精确算法（如分支定界）求解 15 客户 VRP，可能因解空间爆炸耗时**数小时**；而混合策略仅用 **2.5 分钟**就得到高质量解，兼顾实用性和精度。

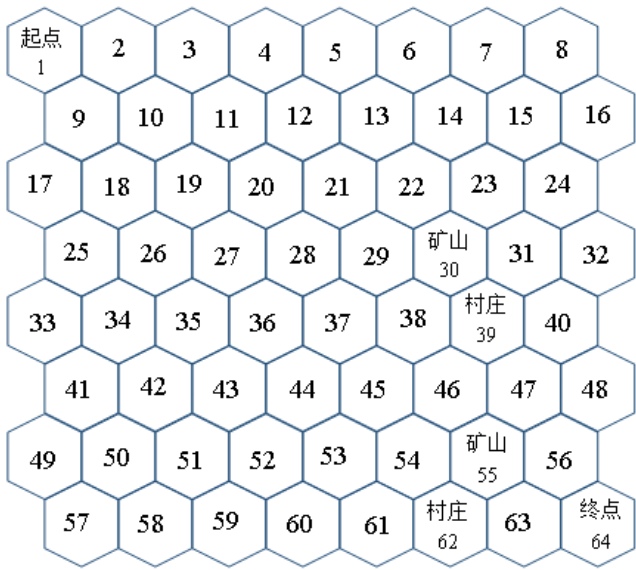


5.5 复杂问题处理技巧之问题拆分 (2020B)

考虑如下的小游戏：玩家凭借一张地图，利用初始资金购买一定数量的水和食物（包括食品和其他日常用品），从起点出发，在沙漠中行走。途中会遇到不同的天气，也可在矿山、村庄补充资金或资源，目标是在规定时间内到达终点，并保留尽可能多的资金。

负重上限		1200千克	初始资金	10000元	
截止日期		第30天	基础收益	1000元	
资源	每箱质量 (千克)	基准价格 (元/箱)	基础消耗量（箱）		
			晴朗	高温	沙暴
水	3	5	5	8	10
食物	2	10	7	6	10

日期	1	2	3	4	5	6	7	8	9	10
天气	高温	高温	晴朗	沙暴	晴朗	高温	沙暴	晴朗	高温	高温
日期	11	12	13	14	15	16	17	18	19	20
天气	沙暴	高温	晴朗	高温	高温	高温	沙暴	沙暴	高温	高温
日期	21	22	23	24	25	26	27	28	29	30
天气	晴朗	晴朗	高温	晴朗	沙暴	高温	晴朗	晴朗	高温	高温



第二关



5.5 复杂问题处理技巧之问题拆分 (2020B)

参考2020B优秀论文B078

➤ 求解最佳策略的弗洛伊德算法

根据上述分析，在不挖矿时，我们直接选择最短路径到达终点，因此我们借助弗洛伊德算法进行求解，具体步骤如下。

第 1 步：使用邻接矩阵，将题目中所给的地图转换为图论中的图，并对距离矩阵进行初始化，能直接到达的结点之间距离为 1，否则为无穷大。

第 2 步：借助弗洛伊德算法求解任意两点之间的最短距离，即对于任意两个结点，若经过另外某个结点“中转”，得到的距离小于原来两结点的距离，则该距离为两点之间更短的距离；考虑将所有结点作为中转点的情况，即可获得两结点之间的最短路径。

第 3 步：初始化天气情况、各种情况下生活必需品的消耗量、最大负重等常量。

第 4 步：根据第 2 步中的结果，我们可以得到从起点到终点的最短路程 d ，根据式(5.1.3)可算出参与者在起点所需购买的生活必需品数量，进而可根据式(5.1.4)算出到达终点时的资金剩余量；使用 Matlab 编程^[3]，可获得相关数据如下表所示。

表 5.1.1：不挖矿时第一关、第二关中参与者在起点购买生活必需品数量及到达时资金剩余量

关卡编号	第一关	第二关
最短路程	3	11
在起点购买食物/箱	38	170
在起点购买水/箱	42	182
到达终点资金剩余量/元	9410	7390
最短路径	1→25→26→27	1→2→3→4→5→13 →22→30→39→47→56→64

● 挖矿的情况，使用回溯算法求解

若参与者选择挖矿，则其可能有多种路线选择方式，如起点→矿山→村庄→终点，起点→村庄→矿山→终点等，甚至可能来回往返村庄、矿山之间以补给资源，因此，我们必须考虑所有可能的情况，并对一些不可能出现的情况进行剪枝，以减少遍历次数，因此我们设计了回溯算法对挖矿的情况进行求解。该算法的精巧之处在于：考虑到村庄补给的情况较为复杂，我们设计的算法没有按照常规的路径去考察在村庄是否需要补给，而是在每一个结点考察是否需要补给，如果不需要，则继续深度优先遍历，否则回溯到上一个村庄的位置进行补给，或增加在起点购买的生活必需品的数量，以达到剪枝的目的；该算法的具体步骤如下，流程图如右图所示，由于该算法是一个递归算法，流程图只能简要的体现递归过程。

表 5.1.2：挖矿时第一关、第二关中参与者在起点购买生活必需品数量及到达时资金剩余量

关卡编号	第一关	第二关
在起点购买食物/箱	333	405
在起点购买水/箱	178	130
到达终点资金剩余量/元	10470	12730

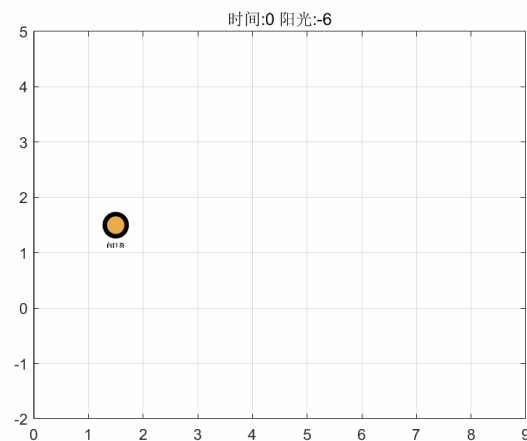
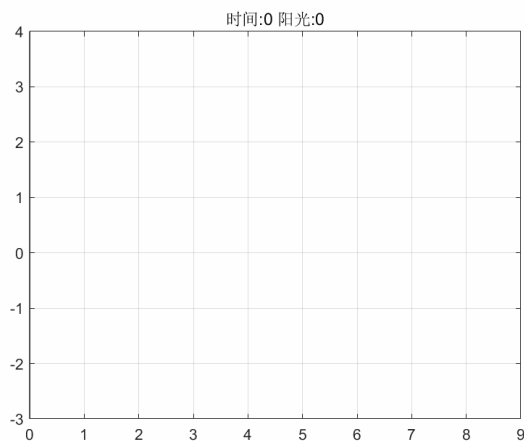
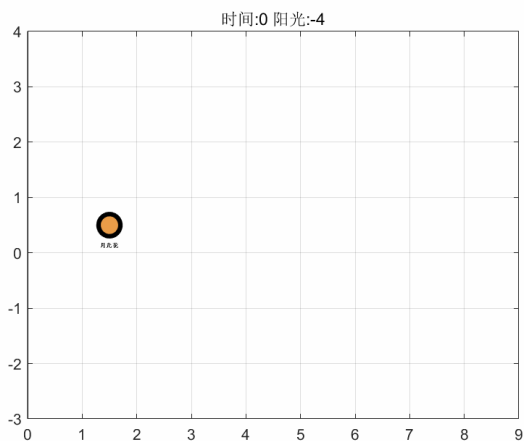


5.6 复杂问题处理技巧之仿真求解 (2022)

题目：可怕的僵尸即将入侵，每种僵尸都有不同的特点，例如铁桶僵尸拥有极强的抗击打能力，暴走僵尸有较快的移动速度等。玩家防御僵尸的方式就是栽种植物。每种植物有不同的功能。玩家可以针对不同僵尸的特性来合理地种植植物，这是胜利的诀窍。最终的目的是通关游戏（消灭所有僵尸，参赛者不必更深入地了解该游戏），并保留尽可能多的阳光。

规则：略（七大规则）

问题1：若场地只有从左至右的9个格，每隔9秒会产生一个僵尸（第一个僵尸出现在第0秒），在游戏时僵尸的出现顺序与种类事先全部已知。试给出一般情况下玩家的最优策略。并求解附件4中的“第一关”和“第二关”。





5.6 复杂问题处理技巧之仿真求解 (2024B)

6.2 求解校准——基于 BF-MC 的模拟仿真

在样本检测值（成品次品率）和样本数（抽样检测的零配件个数）不确定的情况下为了验证上述假设检验模型求解得到的最优检测方案的正确性。

本文首先利用 BF（Brute Force）算法的早停机制确定样本检测值对应的最小样本数，进一步的为了匹配题目给出的两种情形，对样本进行抽样，本文选择基于采样的学习方法类 MC（Monte Corlo）算法。规则图如下所示：

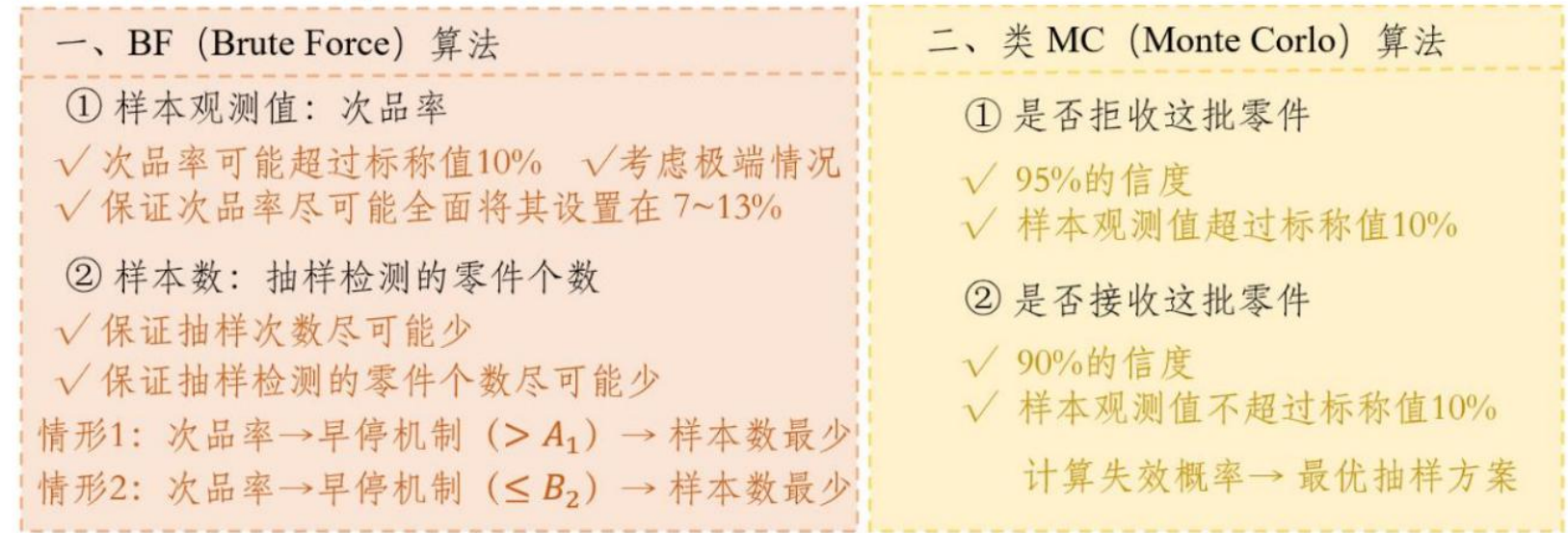


图 6-4 BF-MC 算法规则图

思路：由于零配件抽样检验为重复的独立实验，适用二项分布。传统抽样方案通常样本量较大，导致成本增加。为解决这一问题，本文采用**序贯抽样假设检验**方法，在抽样过程中同步计算，能够有效减少抽样次数，确定不同次品率下的最小样本量。此外，本文通过**BF-MC 模拟仿真对初步求解结果进行了校准**，并进一步分析不同次品率情况下的优化抽样方案。



6.1 基于问题的算法写作（切记不要照抄）

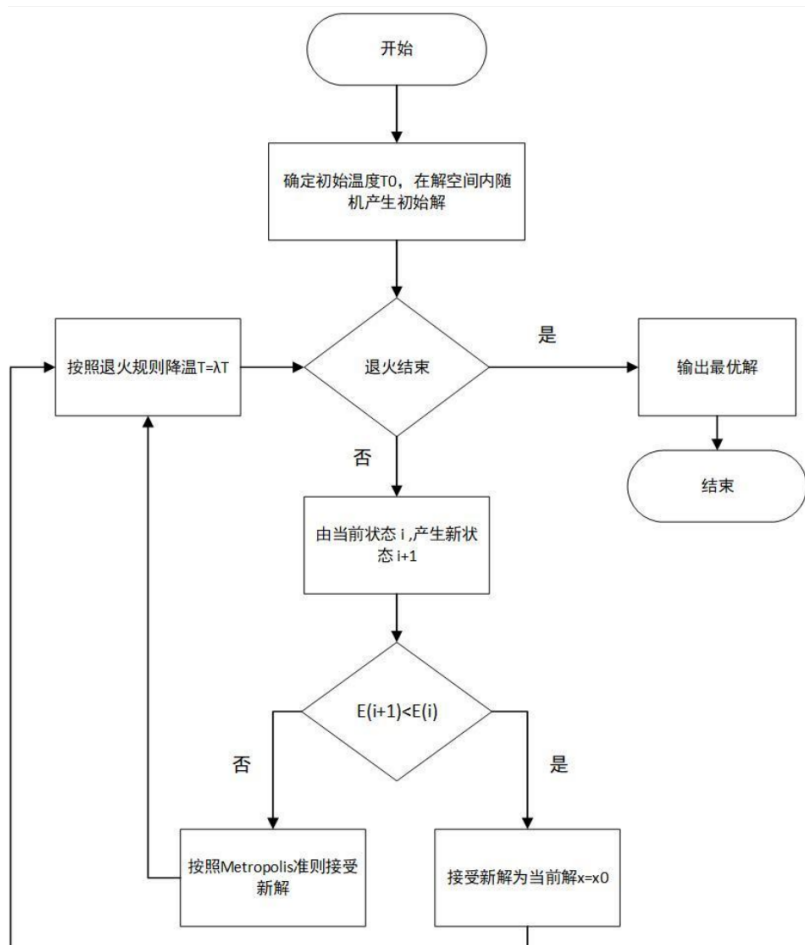


图 9：模拟退火流程图

模拟退火的求解过程流程：

1. 首先，要确定问题的决策变量和它们的取值范围，即问题的解空间。
2. 随机生成一个初始解作为当前解，并设置一个初始温度。
3. 将问题的优化目标转化为一个目标函数，该函数将当前解映射到一个实数，表示解的优劣程度。
4. 根据模拟退火算法的 Metropolis 接受准则，确定接受新解的条件。
5. 通过随机扰动的方式，根据当前解产生一个新解。
6. 计算新解与当前解在目标函数上的差异，即目标函数值的变化。
7. 根据接受准则，判断是否接受新解。如果新解更优秀，则直接接受；如果新解比较差，但根据一定概率接受，以便能够跳出局部最优解。
8. 根据接受准则的结果，更新当前解和温度。可以通过调整参数的取值或者降低温度来逐步收敛到最优解。
9. 设置终止条件，比如达到一定的迭代次数或者目标函数值的变化小于某个阈值时终止优化过程。

（2023A-某国二队伍第二问，这种算法写法不行）



6.2 优秀算法写作展示一

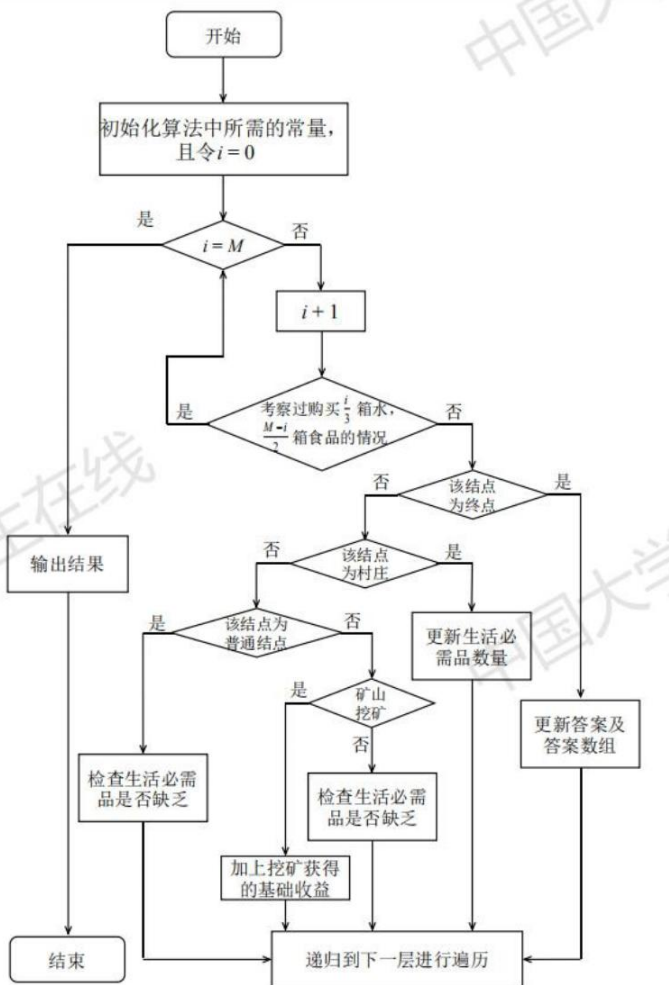


图 5.1.1: 回溯法流程图

5.3.2 基于贪心算法循环遍历确定最优测线布设

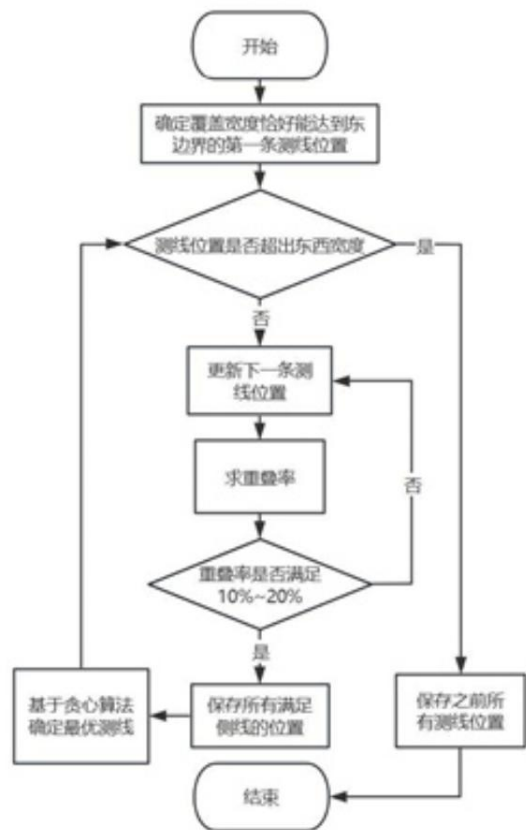


图 17: 最优测线布设求解流程

对于上述过程建立的确定测线位置的单目标优化模型, 我们基于贪心算法的思想逐步优化求解。

Step 1 以矩形海域的东边界为始边, 以覆盖宽度恰好能达到东边界确定出第一条测线的位置。

Step 2 从第一条测线的位置开始, 以 1m 为步长逐步向后遍历确定下一条测线的位置, 求该测线覆盖宽度与前一条测线的重叠率, 将重叠率满足 10%~20% 的测线位置都保存起来, 利用贪心算法优先选择重叠率为 10% 的测线, 从而确定第二条测线的最优位置, 以此类推确定第三条、第四条……测线的位置。

Step 3 依次遍历直到最新一条测线位置超出海域东西宽度, 结束循环, 保存该测线之前的确定的所有测线位置。



6.2 优秀算法写作展示二

6.3.1 算法设计

算法 2: 密集型 Campo 规则布置法计算太阳能塔系统中的日镜场布局

输入: 镜面宽度 $length$, 镜面宽 $width$, 高度 $height$
输出: 各镜面中心坐标 (x, y) , 各镜面尺寸 (长, 宽, 高)

- 1 定义安全半径 $safe_radius = (length + 5)/2$; 定义最大尺寸 max_radius , 当前尺寸 $build_radius$; 定义单圈镜片数量 $N = 2$;
- 2 位置向量初始化 $mirror_x, mirror_y$;
- 3 **while** $build_radius < max_radius$ **do**
- 4 计算半径比 $build_radius$ 大的单圈上能摆放最小镜片数量 N ;
- 5 半径角角度 $theta = \pi/N$; 偏转角度 $angle = 0$;
- 6 **while** $build_radius < max_radius$ **do**
- 7 **for** $i=1$ 至 N **do**
- 8 计算镜片中心坐标:
 $mirror_x+ = build_radius * \cos([0 : 2*theta : 2*\pi - 2*theta] + angle)$;
 $mirror_y+ = build_radius * \sin([0 : 2*theta : 2*\pi - 2*theta] + angle)$;
- 9 **end**
- 10 增加 $angle = angle + theta$;
- 11 计算下一个尺寸 add_radius ;
- 12 **if** $add_radius < safe_radius$ **then**
- 13 **break**
- 14 **end**
- 15 $build_radius+ = add_radius$;
- 16 **end**
- 17 $build_radius+ = 2 * safe_radius$;
- 18 **end**
- 19 镜片长宽高赋值;
- 20 可视化;

算法 3: Particle Swarm Optimization (PSO) 算法

输入: fun : 目标函数, $nvars$: 解向量的维度, lb : 解向量的下界, ub : 解向量的上界
输出: 最优解 x 和最优值 $fun(x)$

- 1 初始化粒子群;
- 2 **foreach** 粒子 $i = 1$ to 粒子数 **do**
- 3 初始化粒子位置 x 和速度 v ;
- 4 初始化个体最优位置 $pbest$ 和个体最优值 $pbest_value$;
- 5 **end**
- 6 初始化群体最优位置 $gbest$ 和群体最优值 $gbest_value$;
- 7 **while** 未达到停止条件或迭代次数未达到最大值 **do**
- 8 **foreach** 粒子 $i = 1$ to 粒子数 **do**
- 9 更新粒子速度和位置;
- 10 根据当前速度和位置计算加速度 a ;
- 11 更新速度:
 $v(i) = w \cdot v(i) + c1 \cdot rand() \cdot (pbest(i) - x(i)) + c2 \cdot rand() \cdot (gbest - x(i))$;
- 12 更新位置: $x(i) = x(i) + v(i)$;
- 13 更新个体最优位置和值;
- 14 **if** $fun(x(i)) < pbest_value(i)$ **then**
- 15 更新 $pbest(i) = x(i)$ 和 $pbest_value(i) = fun(x(i))$;
- 16 **end**
- 17 更新群体最优位置和值;
- 18 **if** $fun(x(i)) < gbest_value$ **then**
- 19 更新 $gbest = x(i)$ 和 $gbest_value = fun(x(i))$;
- 20 **end**
- 21 **end**
- 22 **end**



6.2 优秀算法写作展示三

算法 1: 栅格化模拟定日镜反射太阳光的过程仿真

输入: 待求月份 X 、当地时间 Y 、定日镜以及集热器坐标

输出: 定日镜场每月 21 日的平均光学效率及输出功率

Step1 根据太阳高度角和方位角, 以及当地的地理位置, 输入日期 X 月 21 日和当地时间 Y 时刻, 求解在 X 月 21 日 Y 时刻的太阳光的入射方向, 得到 X 月 21 日 Y 时刻的入射光方向向量。

Step2 根据附件的数据和已知数据可以得到定日镜以及集热器的三维坐标, 由定日镜和集热器的中心可以求出反射光线的方向向量, 入射方向向量和反射方向向量根据菱形几何关系可以求出定日镜法向量。

Step3 开始入射的阴影模拟: 对第 i 个定日镜进行分析, 将该定日镜分成若干栅格, 遍历每个栅格, 对每个栅格执行 Step4。

Step4 将栅格中心作为入射光线的接触点 (即入射光射在栅格中心)。寻找与第 i 个定日镜最近的其他 6 个定日镜, 求出入射光线和这些定日镜所在平面的交点。判断交点是否在这些定日镜框定的矩形内, 若交点落入矩形, 且矩形被入射光穿透 (入射光不是一条直线, 终点在第 i 个定日镜面, 背后的矩形不会被穿透, 需要排除), 则说明入射光被该栅格阻挡, 产生了阴影。若该栅格没有阴影, 转入 Step5 判断该栅格的反射光线是否会被遮挡。

Step5 将栅格中心作为反射光线的出发点。与 Step4 类似, 寻找反射光线和其他 6 个最近邻定日镜的交点。若交点在矩形内, 并被反射光穿透, 则说明产生了遮挡。返回 Step3 遍历其他栅格, 若遍历完成, 记录栅格的阴影遮挡信息, 执行 Step6。

Step6 循环执行 Step3 Step5 直至遍历所有定日镜。遍历完成后执行 Step7 开始模拟截断过程, 计算截断效率。

Step7 开始反射的截断效率计算: 将反射光线模拟成锥形光线, 锥形光线会在集热器上形成光斑。对于第 i 个定日镜, 将该定日镜分成若干栅格, 根据记录的栅格阴影遮挡信息, 得到没有阴影遮挡的栅格所形成的总光斑面积和集热器接收到的光斑面积, 从而计算得出第 i 个定日镜的截断效率。遍历所有定日镜得到所有的截断效率。

Step8 根据以上结果计算光学效率及输出功率。遍历其他日期和时刻, 得到每月 21 日的平均光学效率及输出功率, 从而得到年平均光学效率及输出功率。

8.2.2 基于多目标遗传算法的最优化求解模型构建

由于本问决策变量中既含有 0-1 变量又含有连续变量, 粒子群算法和传统遗传算法的性能较差, 需要使用混合编码和多目标规划的优化算法。本文采用多目标遗传算法 NSGA-II, 并对其加以改进。相对于传统的多目标遗传算法, NSGA-II 引入了非支配排序技术, 能够将个体按照非支配性进行分级, 保留多个非支配解, 找到更多的高质量解; 同时引入了克努森分配距离, 确保前沿中的个体分布更加均匀, 提高了解的多样性 [1]。

算法改进说明:

- **混合编码:** 采用 BG 编码与 RI 编码混合编码以适应本问决策变量既有 01 变量又有连续变量的情况。
- **先验知识:** 融入先验知识, 指定部分初始值, 加快模型收敛。
- **全局存档:** 引入全局存档, 保留全局最优解, 避免优良种群丢失。

(2023A/2023C-优秀论文-自然语言)



6.2 优秀算法写作展示四

算法 2：三分查找算法

输入：单峰曲线离散节点与对应函数值，停止条件 ϵ 为一较小的

输出：极大值和极大值点

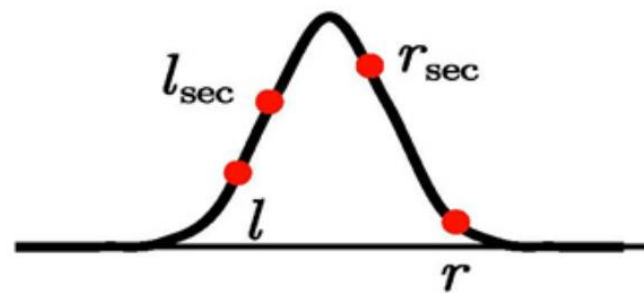


图 7 算法 2 的图形解释

Step1 计算 l_{sec} 和 r_{sec} 的目标函数值 $f(l_{sec})$ 和 $f(r_{sec})$ ，如果 $f(l_{sec}) < f(r_{sec})$ ，转至 Step2；

如果 $f(l_{sec}) > f(r_{sec})$ ，转至 Step3

Step2 极大值一定在 $[l_{sec}, r]$ 内取到，令 $l = l_{sec}$ ，计算更新的 $l_{sec} = l + (r_{sec} - l_{sec})/3$ 和

$r_{sec} = l + 2 \times (r_{sec} - l_{sec})/3$ 。如果 $|l_{sec} - r_{sec}| < \epsilon$ ，结束迭代并输出结果。

Step3 极大值一定在 $[l, r_{sec}]$ 内取到，令 $r = r_{sec}$ ，计算更新的 $l_{sec} = l + (r_{sec} - l_{sec})/3$ 和

$r_{sec} = l + 2 \times (r_{sec} - l_{sec})/3$ 。如果 $|l_{sec} - r_{sec}| < \epsilon$ ，结束迭代并输出结果。



6.2 优秀算法写作展示五

➤ 逐步逼近求解

这里我们采用逐步逼近的方法来求解 θ_2 ，步骤如下：

先将 θ_2 的初始值设定为 θ_1 ，并设定步长 λ 和最小步长 λ_{min} ，进行多次循环， θ_2 每次循环减去一个步长，如果此时 $F(\theta_2)$ 小于 $F(\theta_1) - v(t_2 - t_1)$ ，说明步长过大，则撤回上一步操作（将 θ_2 加上一个步长），并将步长除以 2，缩小步长以更精确地逼近实际值。若步长小于设定的最小步长，说明误差小于最小步长的 2 倍， θ_2 足够接近实际值，则结束循环。

以伪代码的形式呈现如下：

```
逼近求解 $\theta_2$ 伪代码
 $\theta_2 \leftarrow \theta_1, \lambda \leftarrow \lambda_0, \lambda_{min} = 10^{-10}$ 
while  $\lambda \geq \lambda_{min}$ 
   $\theta_2 \leftarrow \theta_2 - \lambda$ 
  if  $F(\theta_2) < F(\theta_1) - v(t_2 - t_1)$ 
     $\theta_2 \leftarrow \theta_2 + \lambda$ 
     $\lambda \leftarrow \lambda/2$ 
  end
end
```

➤ 逐步逼近求解

这里我们采用逐步逼近的方法来求解 $\theta_{i+1,t}$ ，步骤如下：

先将 $\theta_{i+1,t}$ 的初始值设定为 $\theta_{i,t}$ ，并设定步长 λ 和最小步长 λ_{min} ，进行多次循环， $\theta_{i+1,t}$ 每次循环加上一个步长，如果此时两把手间距大于板长，说明步长过大，则撤回上一步操作（将 $\theta_{i+1,t}$ 减去一个步长），并将步长除以 2，缩小步长以更精确地逼近实际值。若步长小于设定的最小步长，说明误差小于最小步长的 2 倍， $\theta_{i+1,t}$ 足够接近实际值，则结束循环。

以伪代码的形式呈现如下：

```
逼近求解 $\theta_{i+1,t}$ 伪代码
 $\theta_{i+1,t} \leftarrow \theta_{i,t}, \lambda \leftarrow \lambda_0, \lambda_{min} = 10^{-10}$ 
while  $\lambda \geq \lambda_{min}$ 
   $\theta_{i+1,t} \leftarrow \theta_{i+1,t} + \lambda$ 
  if  $\theta_{i+1,t}^2 + \theta_{i,t}^2 - 2 \cdot \theta_{i+1,t} \cdot \theta_{i,t} \cdot \cos(\theta_{i+1,t} - \theta_{i,t}) > \frac{d_i^2}{b^2}$ 
     $\theta_{i+1,t} \leftarrow \theta_{i+1,t} - \lambda$ 
     $\lambda \leftarrow \lambda/2$ 
  end
end
```

(2024A-优秀论文-伪代码/类编程语言)



6.2 优秀算法写作展示六

4.5 基于差分进化算法的改进遗传算法的运用

优化算法在解决复杂问题时发挥了重要作用，其中遗传算法与差分进化算法作为两种经典的进化算法，广泛应用于多领域的全局优化问题。本文将遗传算法与差分进化算法相结合，旨在充分发挥二者的优势，提高算法的收敛速度与精度，从而为复杂优化问题的求解提供更为有效的解决方案。

遗传算法（Genetic Algorithm, GA）源于对生物进化过程的模拟，是一种全局优化方法。其核心思想是通过模拟自然选择、遗传和变异等生物进化现象，从初始种群中进行随机选择、交叉和变异，逐步产生更适应环境的个体，使种群在搜索空间中进化到更优区域^[1]。遗传算法的主要步骤包括染色体编码、种群初始化、选择、交叉和变异。它通过群体搜索策略及群体内基因的信息交换，具备较强的全局搜索能力^[2]。然而，遗传算法也存在陷入局部最优解的风险，限制了其在某些复杂优化问题中的效果。

差分进化算法（Differential Evolution, DE）是在遗传算法基础上发展而来的一种多目标优化算法，特别适用于求解多维空间中的全局最优解^[3]。差分进化算法与遗传算法类似，也通过随机生成初始种群，并通过变异、交叉、选择三个步骤优化种群。不同的是，差分进化算法通过父代差分向量生成变异向量，新的个体由交叉操作生成后，再与其父代个体进行选择，从而提高了全局搜索能力^[4]。然而，差分进化算法也可能面临种群停滞和局部最优陷阱等问题。

为了克服遗传算法容易陷入局部最优解的问题，本文将差分进化算法引入遗传算法的框架中。差分进化算法强大的全局寻优能力可以加速遗传算法的收敛速度，并提高其精度。在种群初始化时，采用反向数组法选择适应度较低的种群作为初始个体，进一步提升算法的全局搜索能力和收敛精度^[5]。通过结合两者的优势，本文采用基于差分进化算法的改进遗传算法（DEGA 算法），有效提升了在农作物种植策略优化中实现利润最大化的求解效率，能够获得更优解。基于差分进化算法的改进遗传算法的具体步骤如下：

Step 1 适应度函数

适应度函数用于评估种群优劣程度。在本文的农作物种植策略优化模型中，适应度函数以利润最大化为目标进行设计。适应度值越高，表示该种植策略的预期利润越大，种群越优。在计算个体适应度时，首先对初始化的种群进行处理，基于各类作物的亩均利润及耕地条件，构建三次样条拟合模型，用以拟合作物种植面积与利润之间的关系，生成曲线以得到每一作物在不同耕地条件下的最优种植分配方案，具体为：

$$\begin{cases} X(T), X(T+1), X(T+N) \\ Y(T), Y(T+1), Y(T+N) \\ Z(T), Z(T+1), Z(T+N) \end{cases} \quad (21)$$

利用差分思想设计适应度函数，具体为：

$$\text{Fitness} = \sum_{t=1}^{t=N-1} \sqrt{\Delta X(t) + \Delta Y(t) + \Delta Z(t)} \quad (22)$$

$$\begin{cases} \Delta X(t) = X(T+1) - X(T) \\ \Delta Y(t) = Y(T+1) - Y(T) \\ \Delta Z(t) = Z(T+1) - Z(T) \end{cases} \quad (23)$$

Step 2 种群初始化

首先，根据乡村耕地结构与农作物种植现状及亩均利润的可视化分析结果，初始化 n 个个体 $N_1, N_2, N_3, N_4, N_5, N_6, \dots, N_n$ ，组成初代种群；其次，对种群中每个个体进行三次样条数据插值，生成 n 条路径；然后，对生成的路径点进行变异操作，尽量避免初代种群陷入局部最小值；最后对生成路径的点进行反向反射，以生成反向数，并扩大种群搜索路径范围，形成 $N_1, N_2, N_3, N_4, N_5, \dots, N_{2n}$ 种群。计算适应度时应选取前 n 个适应度较低的个体组成的初始种群。

Step 3 染色体编码

编码即将问题的可行解从其解空间转换到遗传算法所能处理的搜索空间的转换方法。常见编码方法包括二进制编码、格雷码编码、浮点数编码、实数编码。本文采用实数编码方法，对生成种群的个体随机选取待操作染色体 $N_{1, \text{pos}}, N_{2, \text{pos}}, N_{3, \text{pos}}, N_{4, \text{pos}}, \dots, N_{n, \text{pos}}$ 。

Step 4 变异操作

在改进算法中，变异操作即选择差分进化算法变异方式对种群中的每个个体进行变异操作，并采用最佳向量作为当前基向量的变异策略，以加快种群的收敛速度，保证种群多样性。通过变异操作生成新种群，具体公式为：

$$P_N(i) = N_{\text{best}}(i) + F \times [N_1(i) - N_2(i)] + F \times [N_3(i) - N_4(i)] \quad (24)$$

式中， $P_N(i)$ 为对应变异后生成的个体； $N_{\text{best}}(i)$ 为当前迭代次数下的种群中最佳个体； F 为缩放因子； $N_1(i), N_2(i), N_3(i), N_4(i)$ 为种群中互不相同的个体。

Step 5 交叉操作

在改进算法中，交叉操作即对变异后产生的种群中的个体按一定概率对染色体中的基因进行交叉，通过交叉操作生成新种群，具体公式为：

$$C_{N_x}(i) = \begin{cases} P_{N_x}(i), & \text{if } \text{rand}(0,1) \leq CR \text{ or } i = i_{\text{rand}} \\ N_x(i), & \text{else} \end{cases} \quad (25)$$

式中， $C_{N_x}(i)$ 为生成的第 i 个体； CR 为种群交叉概率，设置 $i = i_{\text{rand}}$ 以提高个体之间交叉概率， i_{rand} 为 $[0, n]$ 之间的随机数。

Step 6 选择操作

改进算法中，选择操作即采用遗传算法中的轮盘赌方式整合变异后生成的种群与交叉之后的种群，以形成新种群，并对其按照轮盘赌方式进行选择，选出 $n/2$ 个个体。轮盘赌又称比例选择法，其基本思想为：每个个体被选中的概率与其适应度大小成正比。

首先，计算群体中每个个体的适应度 $\text{Fitness}(N_1), \text{Fitness}(N_2), \text{Fitness}(N_3), \dots, \text{Fitness}(N_n)$ ，其次，计算每个个体被遗传到下一代群体中的概率，具体计算公式为：

$$P(N_i) = \frac{\text{Fitness}(N_i)}{\sum_{j=1}^n \text{Fitness}(N_j)} \quad (26)$$

然后，计算每个个体的累积概率，具体计算公式为：

$$q_i = \sum_{j=1}^i P(N_j) \quad (27)$$

最后，在 $[0,1]$ 区间内产生一个随机数 r ，若 $r < q[1]$ ，则选择个体1，否则选

择个体 k ，使得 $q[k-1] < r < q[k]$ 成立。重复上述步骤直至循环结束。

Step 7 新种群组合方式

对交叉操作后生成的种群中的个体按适应度排序，并选取前 $n/2$ 个个体与选择操作后生成的 $n/2$ 个个体组成含有 n 个个体的新种群，以避免因采用轮盘赌方法而舍弃部分较优个体和算法本身存在的不稳定性。基于差分进化算法的改进遗传算法的流程如图8所示：

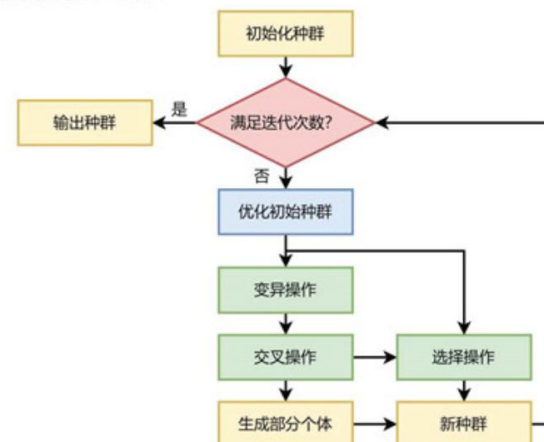


图8 基于差分进化算法的改进遗传算法的基本步骤



数学建模暑期集训

Thank you!